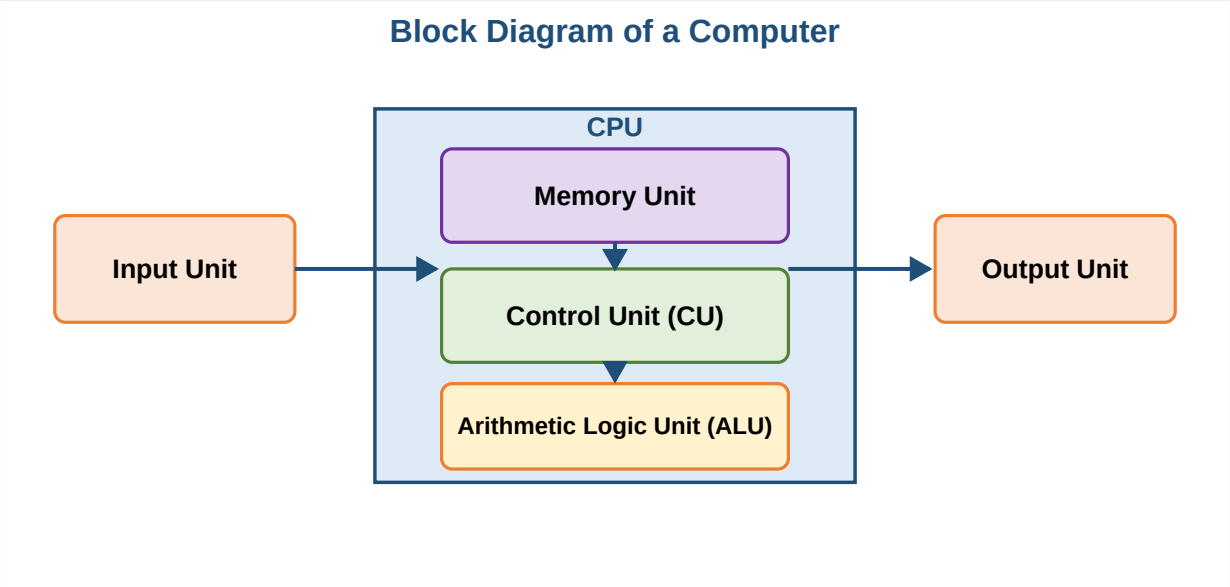
PPS

Programming for Problem Solving

Important Questions & Answers

A Beginner Friendly Guide



Every answer is explained in plain English, supported by a colored diagram, and every C program is walked through step-by-step so beginners never feel lost.

Table of Contents

- 6. C Program : Multiplication of Two 3x3 Matrices
- 7. Strings - Declaration and Initialization
- 8. String Handling Functions (strcpy, strcat, strrev, ...)
- 9. C Program : Palindrome String Check

UNIT 5 - Pointers, Structures and Unions

- 1. Pointers - Declaration, Initialization and Use
- 2. Pointers with Arrays
- 3. C Program : Sum of Array Elements using Pointers
- 4. Structures - Declaration, Initialization and Access
- 5. Array of Structures
- 6. Structure vs Union (with Example)

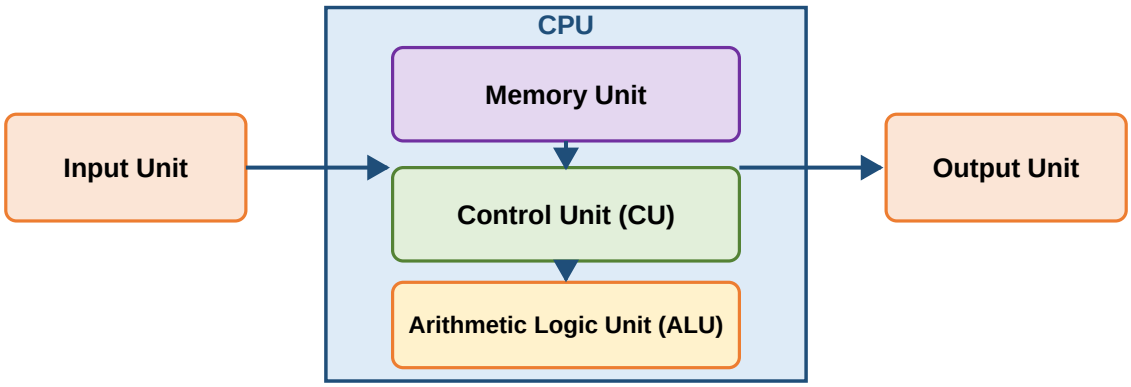
UNIT 1 - Basics of Computing & C Language

Q1. Explain the various functional units of a computer system

Simple idea (for beginners)

A computer is like a small factory. **Input** brings raw materials, the **CPU** processes them, **memory** remembers stuff, and the **output** ships the finished product.

Block Diagram of a Computer



Five things a computer always does:

1. Accepts data or instructions using an **Input unit**.
2. Stores the data inside its **Memory unit**.
3. Processes the data as needed (**ALU** does the maths/logic).
4. Produces results using the **Output unit**.
5. Controls every step with the **Control unit**.

Short description of each unit

Input Unit: Brings data into the computer (keyboard, mouse, scanner, microphone).

Central Processing Unit (CPU): The brain. It contains the ALU and the CU.

Control Unit (CU): Decides *when* to receive, process, and output data. Steps through instructions one at a time.

Arithmetic Logic Unit (ALU): Performs addition, subtraction, multiplication, division, comparison and logical operations.

Memory Unit: Stores data and instructions while they are being used.

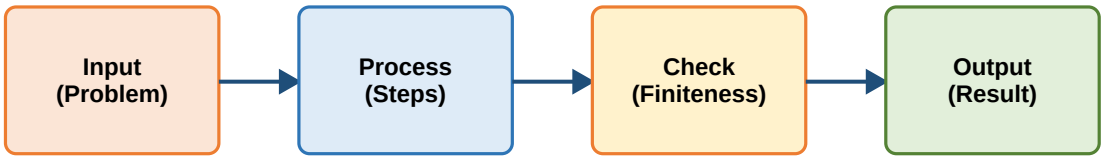
Output Unit: Sends results to the user (monitor, printer, speakers).

Q2. What is an Algorithm? Explain its Properties with an Example

Meaning in one line

An **algorithm** is a **step-by-step recipe** to solve a problem - just like making tea: boil water, add tea leaves, add sugar, pour. Computers love recipes because they can follow them without mistakes.

Algorithm - Step by Step Process



5 Properties of a Good Algorithm:

The 5 properties every algorithm must have

Input: It may take zero, one, or more inputs.

Output: It must produce at least one output.

Finiteness: It must end after a fixed number of steps (no infinite loops).

Definiteness: Each step must be clear - no ambiguity, no errors.

Effectiveness: Each operation must be simple enough to be done in a finite time.

Example : Largest of 3 numbers

Algorithm

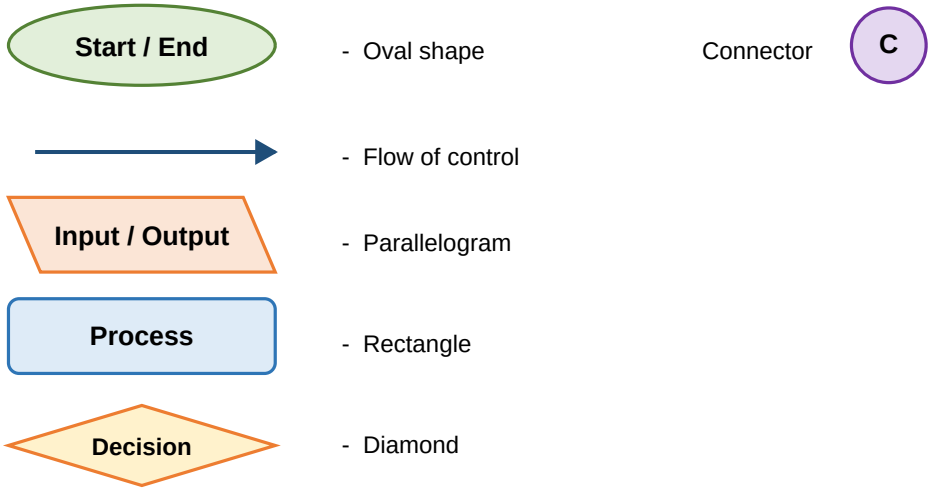
```
Step 1: Start
Step 2: Declare the variables A, B, C
Step 3: Read the values of A, B, C
Step 4: If (A > B) and (A > C) then print "A is largest"
Step 5: else if (B > C) print "B is largest"
Step 6: else print "C is largest"
Step 7: Stop.
```

Q3. What is a Flowchart? Explain the symbols with an example

Plain language

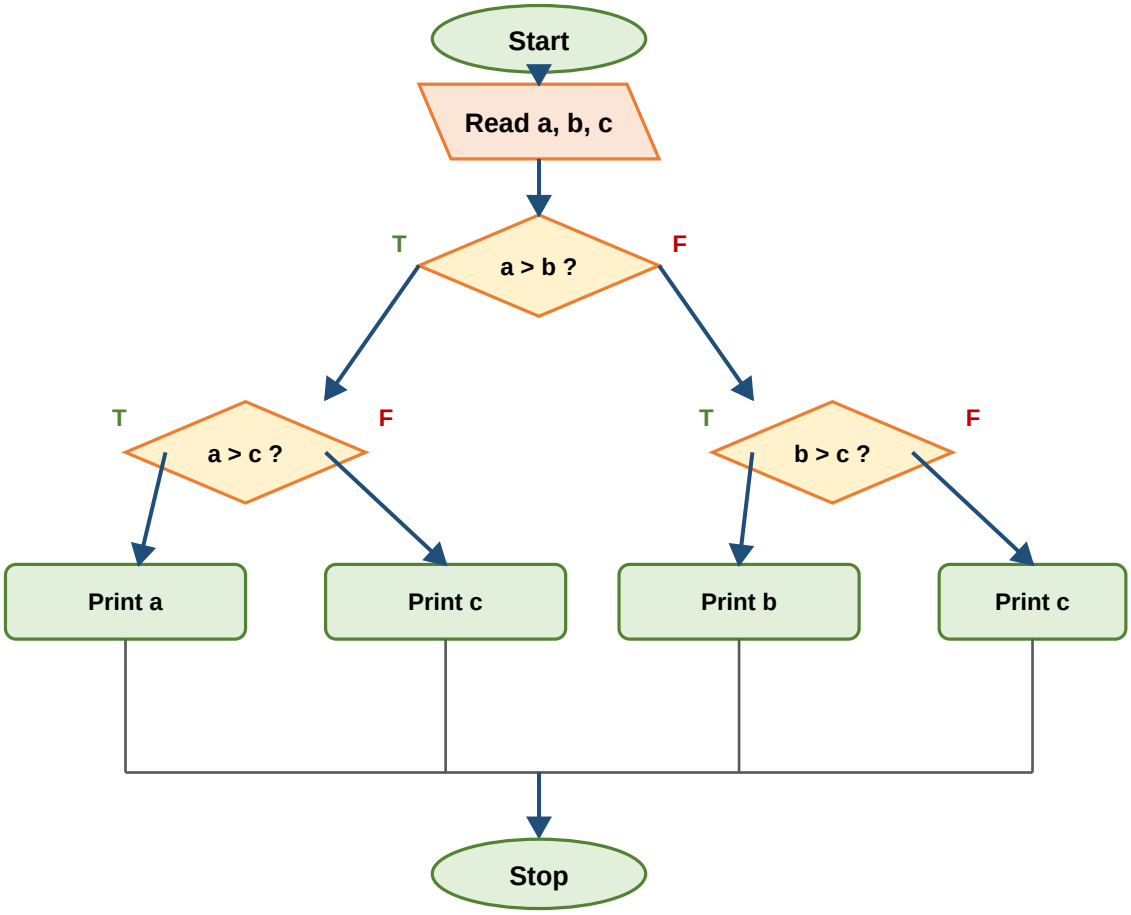
A **flowchart** is a **picture** of an algorithm - boxes for steps and arrows for flow. Pictures are easier to follow than text, especially for beginners.

Flowchart Symbols Reference



Example : Largest of 3 numbers (flowchart)

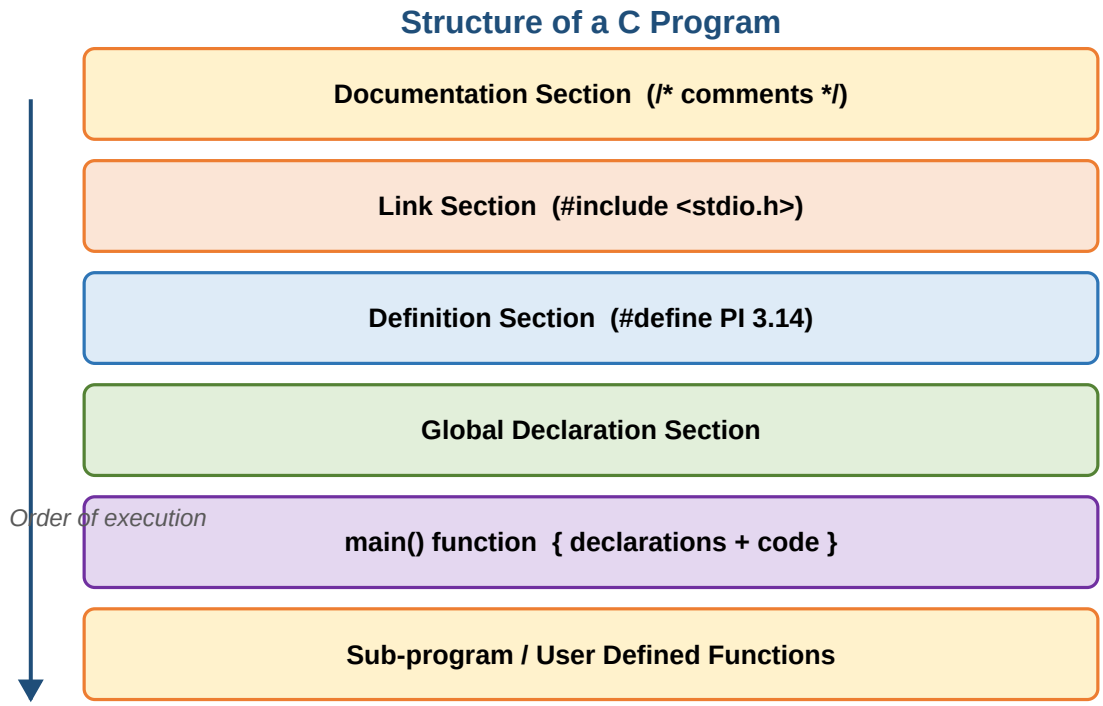
Flowchart: Largest of 3 Numbers



Q4. Explain the Structure of a C Program

Think of it as a layered cake

Every C program is made of 6 layers. You do not need all of them every time, but they always appear in this order.



Documentation Section: Comment lines describing the program. Example: `/* Simple Interest Program */`

Link Section: Tells the compiler to link header files. Example: `#include <stdio.h>`

Definition Section: Declares symbolic constants using `#define`. Example: `#define PI 3.14`

Global Declaration Section: Declares variables used by more than one function.

main() Section: Every program starts here. It has two parts - declaration part (variables) and executable part (statements).

Sub-program Section: User defined functions placed before or after main().

Tiny example

Skeleton C Program

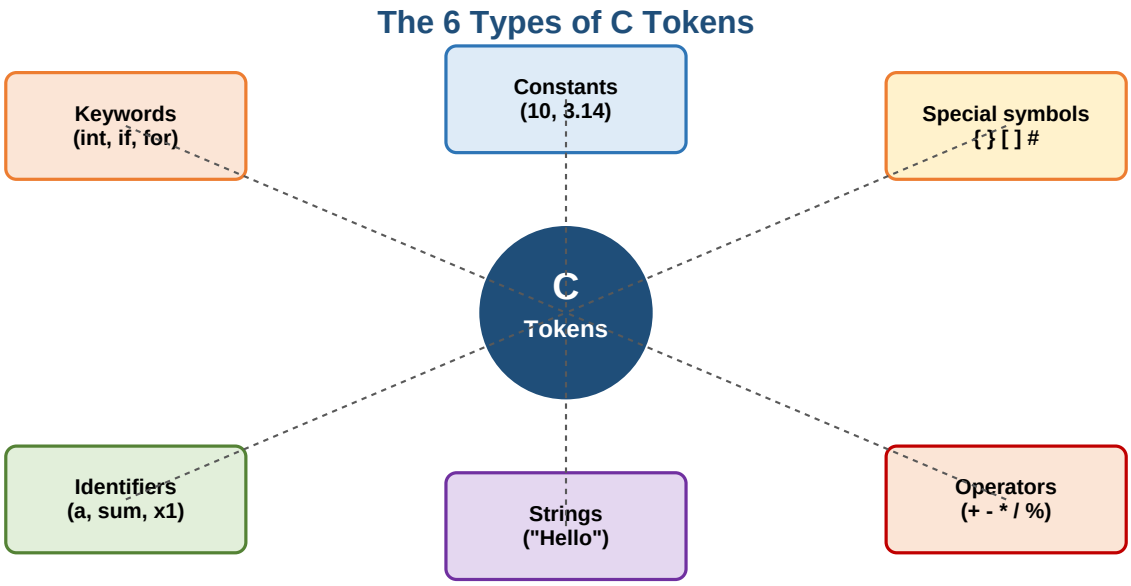
```
/* Documentation: First C program */
#include <stdio.h>          // Link section
#define PI 3.14              // Definition section
int g = 5;                   // Global declaration

int main()                   // main() section
{
    int r = 2;               // declaration part
    float area;
    area = PI * r * r;       // executable part
    printf("Area = %f", area);
    return 0;
}
```

Q5. Define C Tokens and explain them in detail

Meaning

A **token** is the **smallest unit** of a C program - the Lego bricks from which we build any program. C has exactly 6 kinds of tokens.



The 6 kinds with examples

Keywords: Reserved words with a fixed meaning. C has 32 keywords like `int`, `float`, `if`, `else`, `for`, `while`, `switch`, `return`. **Rules:** lowercase only, cannot be used as variable names.

Identifiers: Names we give to variables, functions and arrays. Ex: `a`, `sum`, `total1`. **Rules:** start with letter or _, no spaces, no keywords, case-sensitive, max 31 chars.

Constants / Literals: Fixed values that never change during execution. Integer (10), real (3.14), single char ('A'), string ("Hello").

Strings: Group of characters inside double quotes. Ex: `"MRCET"`.

Special symbols: Punctuation used by the language: `{ } ( ) [ ] , ; # $ @`.

Operators: Symbols that do work on values. Arithmetic, logical, relational, etc. (see Q7).

32 Keywords of C

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile

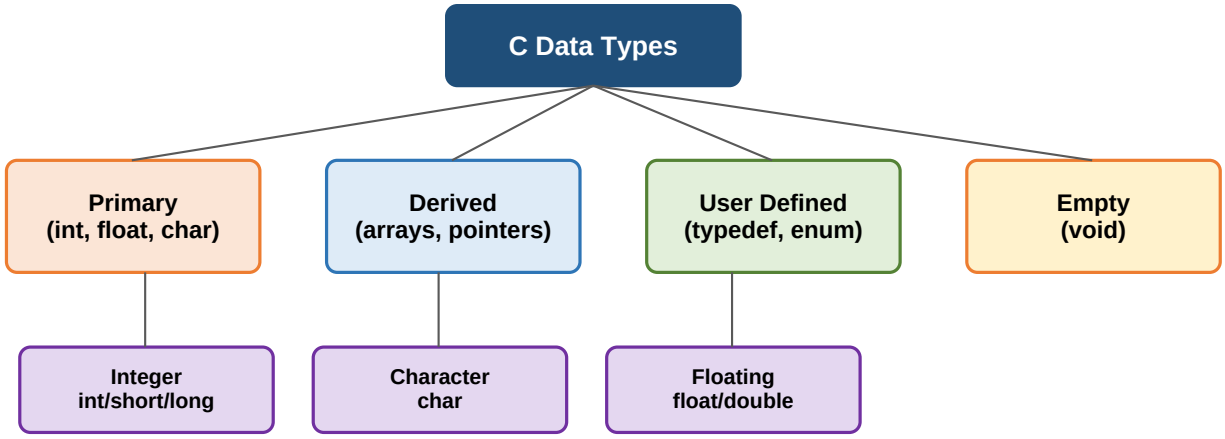
do	if	static	while

Q6. Explain all Primary / Fundamental Data Types in C

Why data types?

A data type tells C **what kind of value** can be stored and **how much memory** it needs. Picking the right type saves memory and prevents wrong results.

C Data Types - Family Tree



int = 2 bytes | char = 1 byte | float = 4 bytes | double = 8 bytes

Primary data types summary table

Type	Keyword	Size	Range	Format
Integer	int	2 B	-32,768 to 32,767	%d
Short int	short	1 B	-128 to 127	%d
Long int	long	4 B	-2,147,483,648 to 2,147,483,647	%ld
Character	char	1 B	-128 to 127	%c / %s
Float	float	4 B	3.4E-38 to 3.4E+38	%f
Double	double	8 B	1.7E-308 to 1.7E+308	%lf
Long double	long double	10 B	3.4E-4932 to 1.1E+4932	%Lf

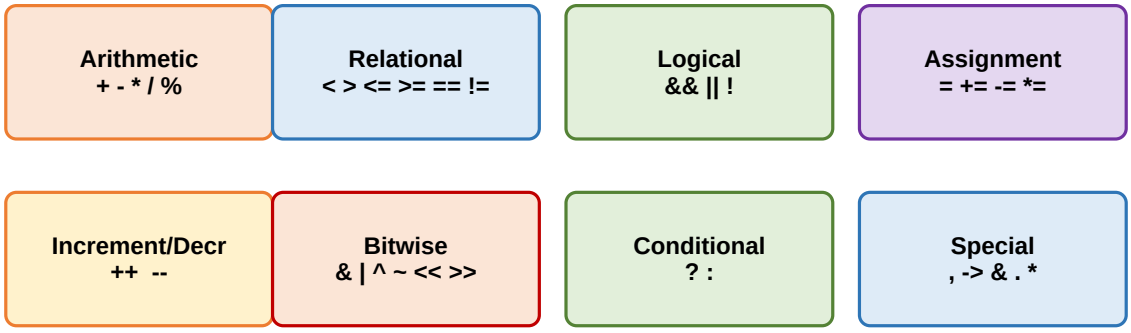
Tip: *use the smallest type that fits your value - this saves memory on low-resource devices.*

Q7. Explain different types of Operators in C

What is an operator?

An **operator** is a symbol that tells the computer to do something with one or more values (called **operands**). Example: in a + b, + is the operator, a and b are operands.

Types of Operators in C



1. Arithmetic + - * / % - basic maths. / gives quotient, % gives remainder (works on integers only).

2. Relational < > <= >= == != - compares two values. Returns 1 (true) or 0 (false).

3. Logical && || ! - combines conditions. AND, OR, NOT.

4. sizeof() Gives the number of bytes a type or variable uses. `sizeof(int)` = 2 (or 4 on 32-bit).

5. Assignment = += -= *= /= %= - stores a value into a variable. a += 5 means a = a + 5.

6. Conditional / Ternary `exp1 ? exp2 : exp3` - a one-line if-else. If exp1 is true → run exp2, else run exp3.

7. Increment / Decrement ++ and --. Pre-increment updates first, then assigns (++a). Post-increment assigns first, then updates (a++).

8. Bitwise & | ^ ~ << >> - works directly on bits (0s and 1s). Cannot be used with real numbers.

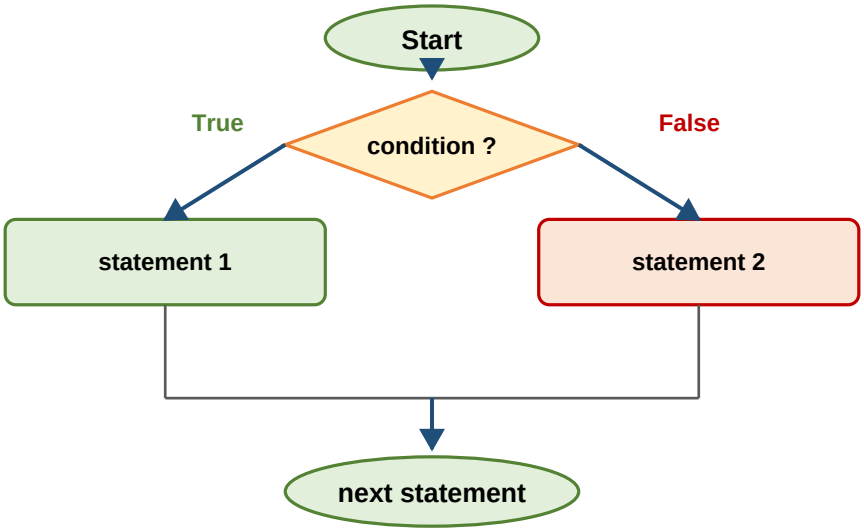
9. Special , comma, -> arrow (for structure pointers), & address-of, . dot (member access), * value-at-address.

UNIT 2 - Control Statements and Programs

Q1. Conditional / Decision Making / Selection Statements in C

Conditional statements help the program **choose a path** based on a condition, just like deciding "if it rains, I take an umbrella, else I do not".

if-else - Two Way Branching



1. if statement (one-way)

Syntax

```
if (condition)
{
    statements;
}
next statement;
```

If the condition is **true**, the block is executed, then the program continues. If **false**, the block is skipped.

2. if-else statement (two-way)

Syntax

```
if (condition)
{
    statement1;
}
else
{
    statement2;
}
```

3. nested if-else (multi-way)

Syntax

```
if (condition1)
{
    if (condition2) { statement1; }
    else            { statement2; }
}
else
{
    statement3;
}
```

4. switch statement (many-way)

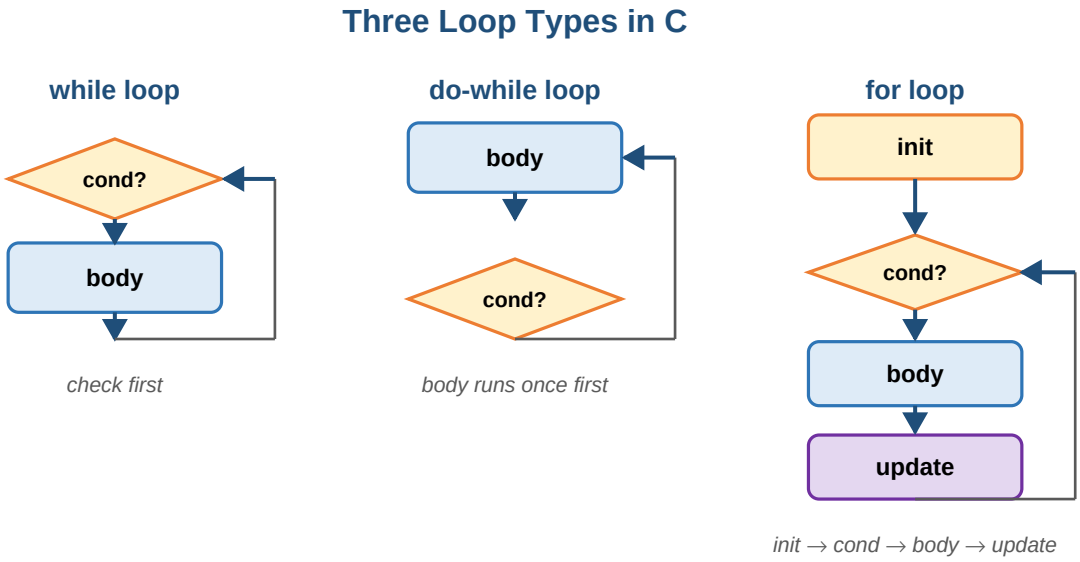
Syntax

```
switch(expression)
{
    case value1:  block1;  break;
    case value2:  block2;  break;
    ...
    default:      default_block;
}
```

The value of `expression` *is matched against each* `case`*. If nothing matches,* `default` *runs.*

Q2. Loops / Repetition / Iterative Statements in C

Loops repeat a block of code **many times** until a condition becomes false. C has three loop types:



1. while loop (entry controlled)

Syntax

```
while (condition)
{
    body;
}
next statement;
```

First checks the condition, then enters the body. Minimum runs = 0.

2. do-while loop (exit controlled)

Syntax

```
do
{
    body;
} while (condition);
next statement;
```

Runs the body first, then checks. Minimum runs = 1.

3. for loop (entry controlled)

Syntax

```
for (init ; condition ; update)
{
    body;
}
```

Best loop when you know how many times to repeat.

Q3. Unconditional / Jump Statements in C

These statements **change the normal flow** of a program by jumping somewhere else directly.

goto: Jumps to a labelled statement. `goto label; ... label: statement;`

break: Exits the loop or switch immediately and continues with the next statement after it.

continue: Skips the rest of the body and goes to the next iteration of the loop.

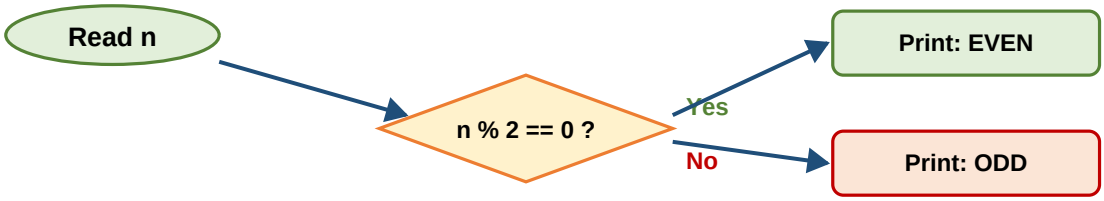
Beginner advice: *avoid* `goto`*. Use* `break` *and* `continue` *inside loops where they really help readability.*

Q4. C program to check whether the given number is Even or Odd

The idea

Divide the number by 2 and look at the remainder. If remainder is 0, the number is **even**; otherwise it is **odd**. We use the **modulus operator** % which gives the remainder.

Even or Odd - Logic Map



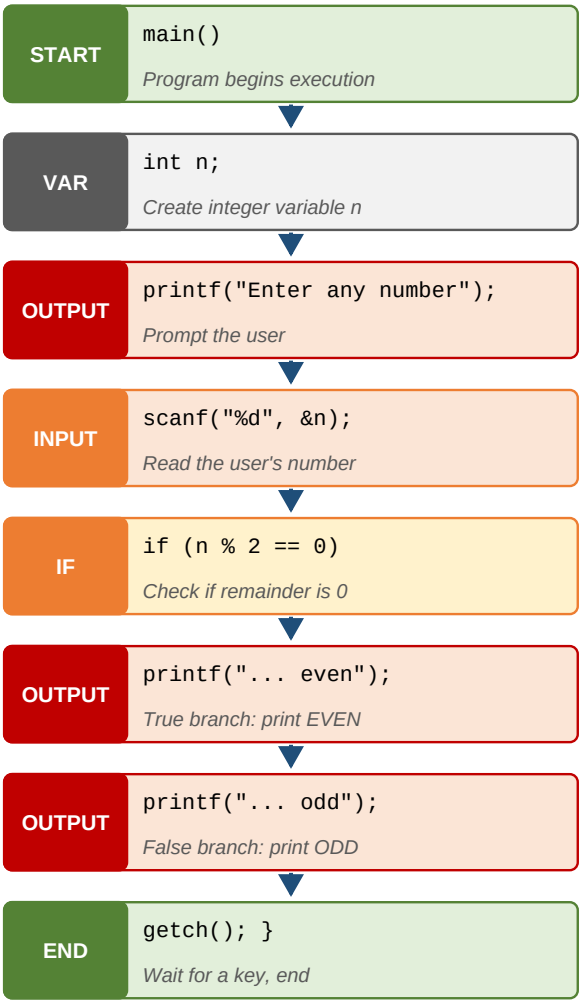
Key idea: use MOD (%) to find the remainder when divided by 2.

remainder 0 → EVEN | remainder 1 → ODD

Program

```
#include <stdio.h>
#include <conio.h>
main()
{
    int n;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("The given number is even");
    else
        printf("The given number is odd");
    getch();
}
```

Instruction-by-instruction block diagram



Step-by-step explanation

Step 1. Include the standard I/O header to use printf and scanf.

Step 2. Declare an integer variable **n**.

Step 3. Ask the user to enter a number, read it with scanf.

Step 4. Check n `%` 2. If the remainder is 0 print *even*, else print *odd*.

Step 5. `getch()` keeps the output window open until the user presses a key (Turbo C).

Q5. C program to find the Roots of a Quadratic Equation

A quadratic equation looks like $ax^2 + bx + c = 0$. The two roots are calculated using the formula:

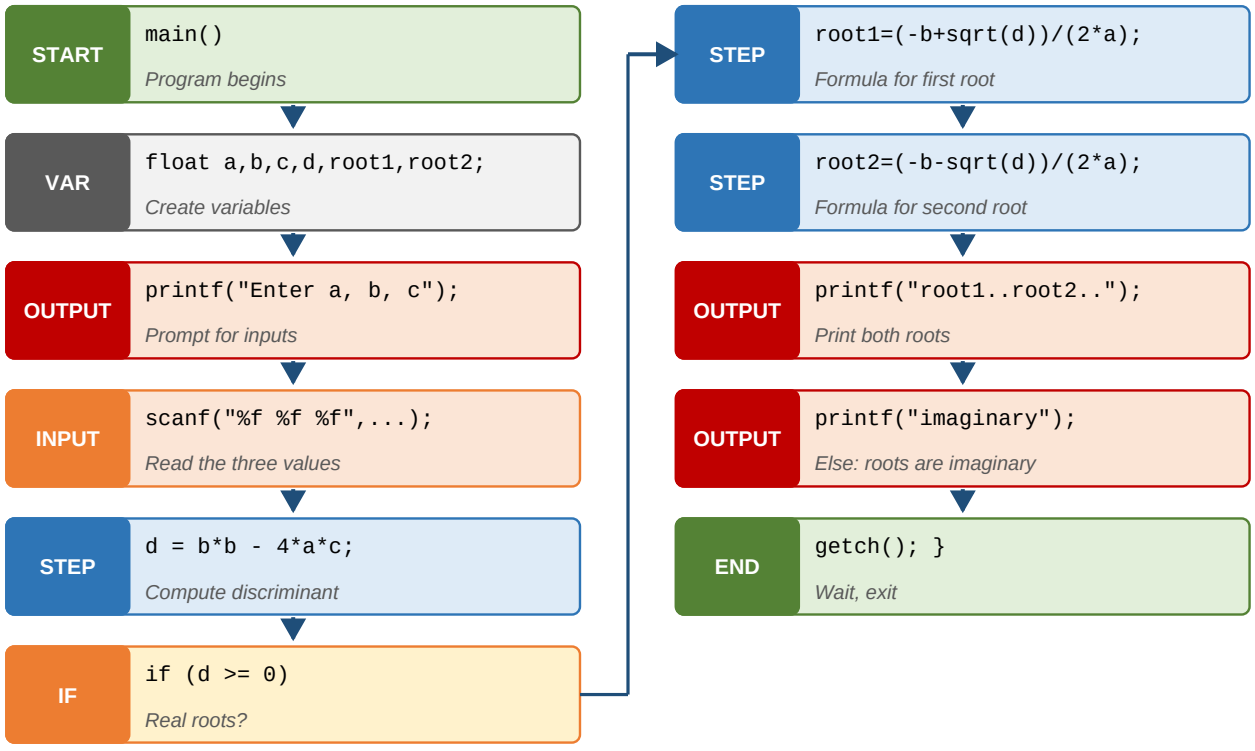
$$x = (-b \pm \sqrt{(b^2 - 4ac)}) / (2a)$$

Discriminant (d) = `b*b - 4*a*c`. If `d >= 0` the roots are real, otherwise they are imaginary.

Program

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
void main()
{
    float a, b, c, d, root1, root2;
    clrscr();
    printf("Enter the values of a, b, c\n");
    scanf("%f %f %f", &a, &b, &c);
    d = (b * b) - (4 * a * c);
    if (d >= 0)
    {
        root1 = (-b + sqrt(d)) / (2 * a);
        root2 = (-b - sqrt(d)) / (2 * a);
        printf("root1 = %f, root2 = %f", root1, root2);
    }
    else
        printf("Roots are imaginary");
    getch();
}
```

Instruction-by-instruction block diagram



Step-by-step explanation

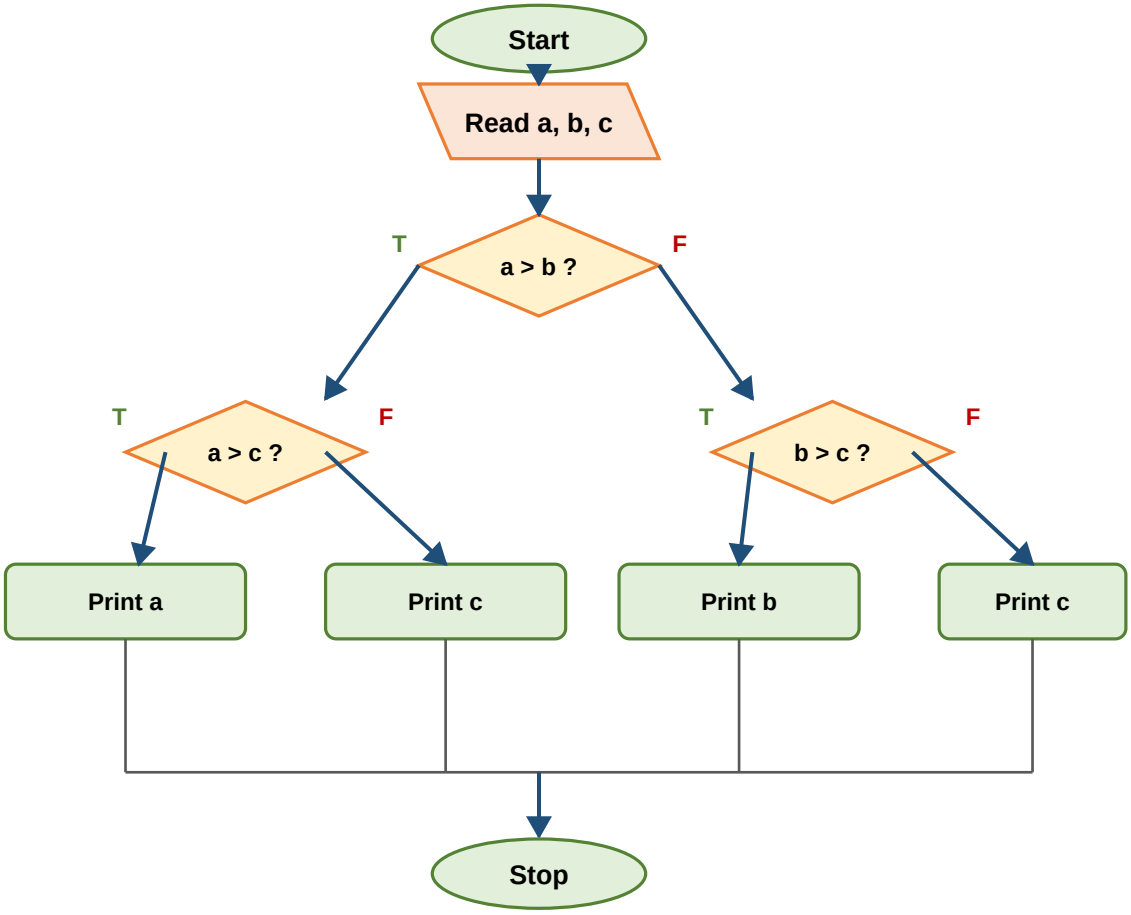
Step 1. Include `math.h` to use `sqrt()`.
Step 2. Declare float variables for coefficients and roots.
Step 3. Read the three coefficients a, b, c.
Step 4. Compute discriminant `d = b*b - 4*a*c`.
Step 5. If `d >= 0` apply the formula and print both roots.
Step 6. Otherwise print *Roots are imaginary*.

Q6. C program to find the Largest number among 3 Numbers

The idea

Compare the three numbers in pairs using nested if-else to find the largest. See the same logic as a picture below:

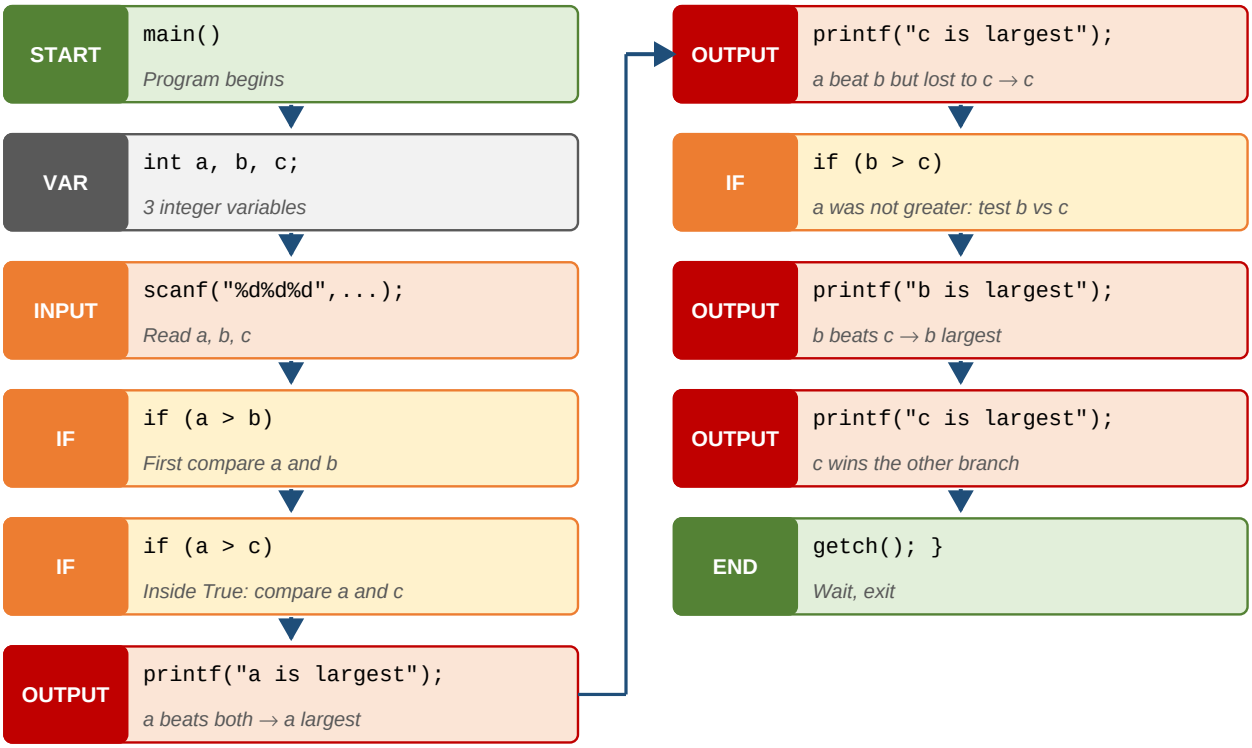
Flowchart: Largest of 3 Numbers



Program

```
#include <stdio.h>
#include <conio.h>
main()
{
    int a, b, c;
    clrscr();
    printf("Enter the values of a, b, c\n");
    scanf("%d %d %d", &a, &b, &c);
    if (a > b)
    {
        if (a > c) printf("a is largest number");
        else       printf("c is largest number");
    }
    else
    {
        if (b > c) printf("b is largest number");
        else       printf("c is largest number");
    }
    getch();
}
```

Instruction-by-instruction block diagram

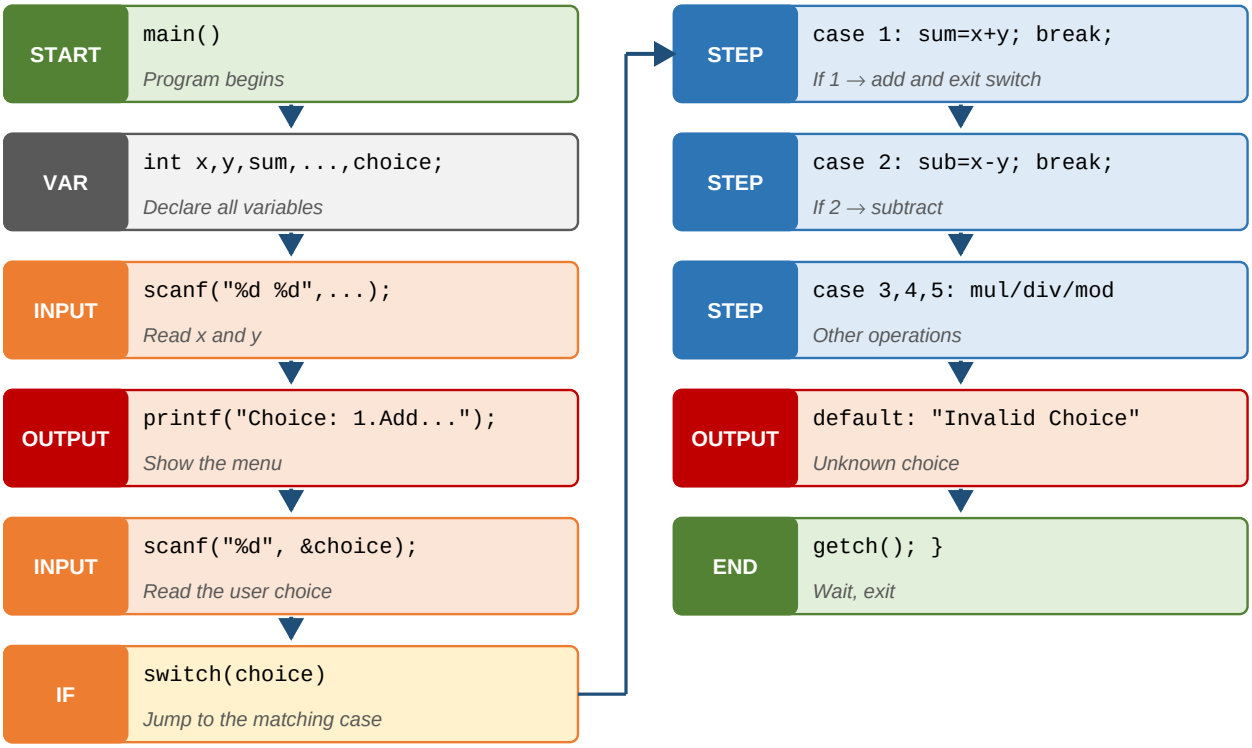


Q7. C program to perform all Arithmetic operations using switch

Program

```
#include <stdio.h>
#include <conio.h>
main()
{
    int x, y, sum, sub, mul, div, mod, choice;
    clrscr();
    printf("Enter the value of x and y\n");
    scanf("%d %d", &x, &y);
    printf("Choice:\n 1. Add\n 2. Sub\n 3. Mul\n 4. Div\n 5. Mod\n");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1: sum = x + y; printf("Sum = %d", sum); break;
        case 2: sub = x - y; printf("Sub = %d", sub); break;
        case 3: mul = x * y; printf("Mul = %d", mul); break;
        case 4: div = x / y; printf("Div = %d", div); break;
        case 5: mod = x % y; printf("Mod = %d", mod); break;
        default: printf("Invalid Choice"); break;
    }
    getch();
}
```

Instruction-by-instruction block diagram



Step-by-step explanation

Step 1. Read two integers **x** and **y**.

Step 2. Show a menu and read the user's **choice**.

Step 3. `switch(choice)` jumps to the matching case and runs that block.

Step 4. `break;` exits the switch so the next case does not accidentally run.
Step 5. `default` handles values not in the menu.

Q8. C program to find the Sum of Individual Digits

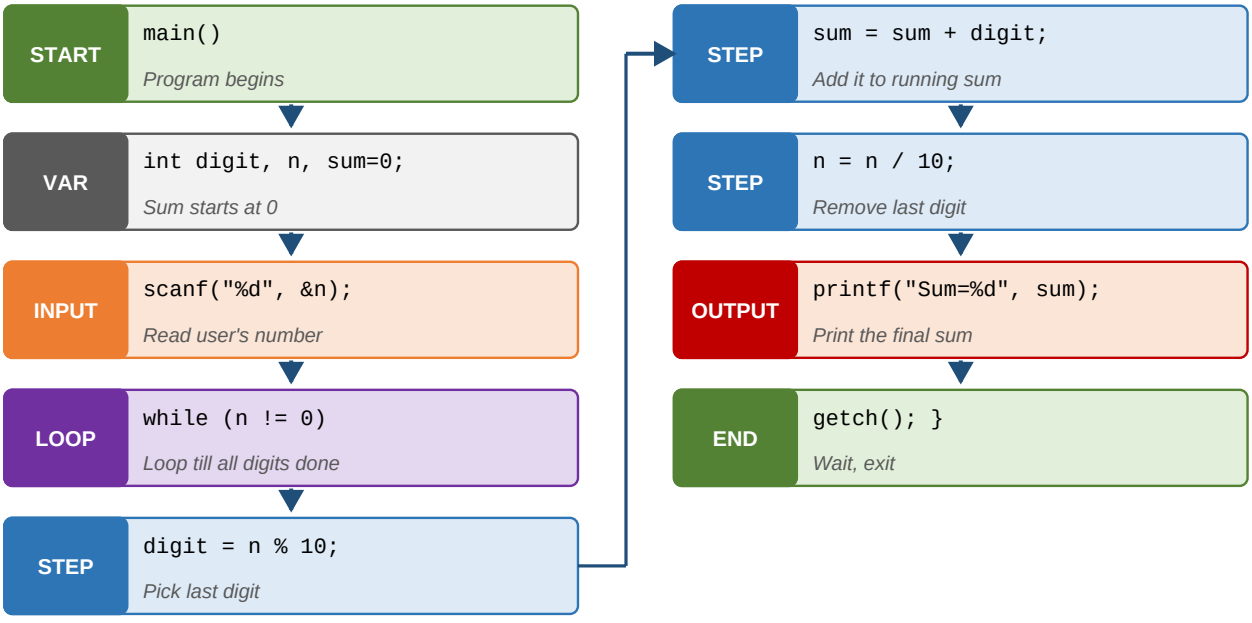
Sum of Digits of 1234 (step by step)

n	digit = n % 10	sum = sum + digit
1234	4	4
123	3	7
12	2	9
1	1	10
0	-	10 (stop)

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int digit, n, sum = 0;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    while (n != 0)
    {
        digit = n % 10;          // pick the last digit
        sum   = sum + digit;     // add it to sum
        n     = n / 10;          // remove the last digit
    }
    printf("Sum of digits = %d", sum);
    getch();
}
```

Instruction-by-instruction block diagram



Trick: `n % 10` *always gives the last digit, and* `n / 10` *removes that digit. Repeat until n becomes 0.*

Q9. C program to find the Reverse of a Given Number

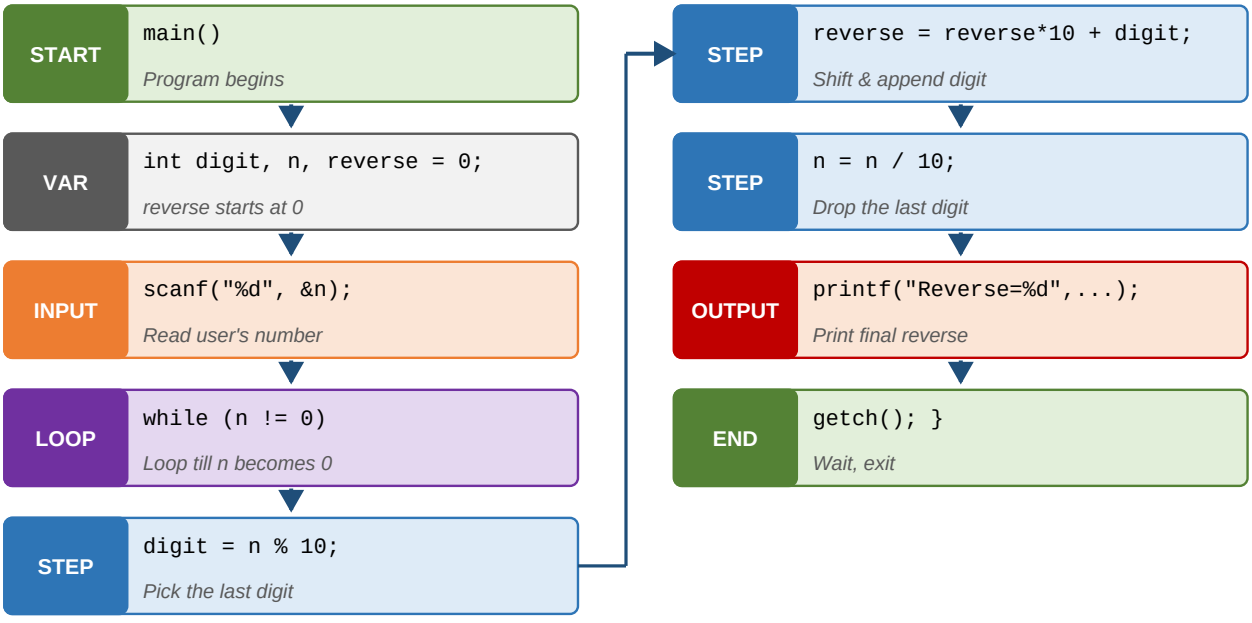
Reverse of 1234 (trace)

n	digit	reverse = reverse*10 + digit
1234	4	0 → 4
123	3	4 → 43
12	2	43 → 432
1	1	432 → 4321
0	-	4321

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int digit, n, reverse = 0;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    while (n != 0)
    {
        digit   = n % 10;                // last digit
        reverse = reverse * 10 + digit;  // shift + append
        n       = n / 10;                // drop last digit
    }
    printf("Reverse = %d", reverse);
    getch();
}
```

Instruction-by-instruction block diagram



Q10. C program to check whether a number is Armstrong

An **Armstrong number** is one whose sum of cubes of its digits equals the number itself. Example: $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$.

Armstrong Check for 153 (n = 153)

Step 1: digits = 1, 5, 3

Step 2: cubes = 1, 125, 27

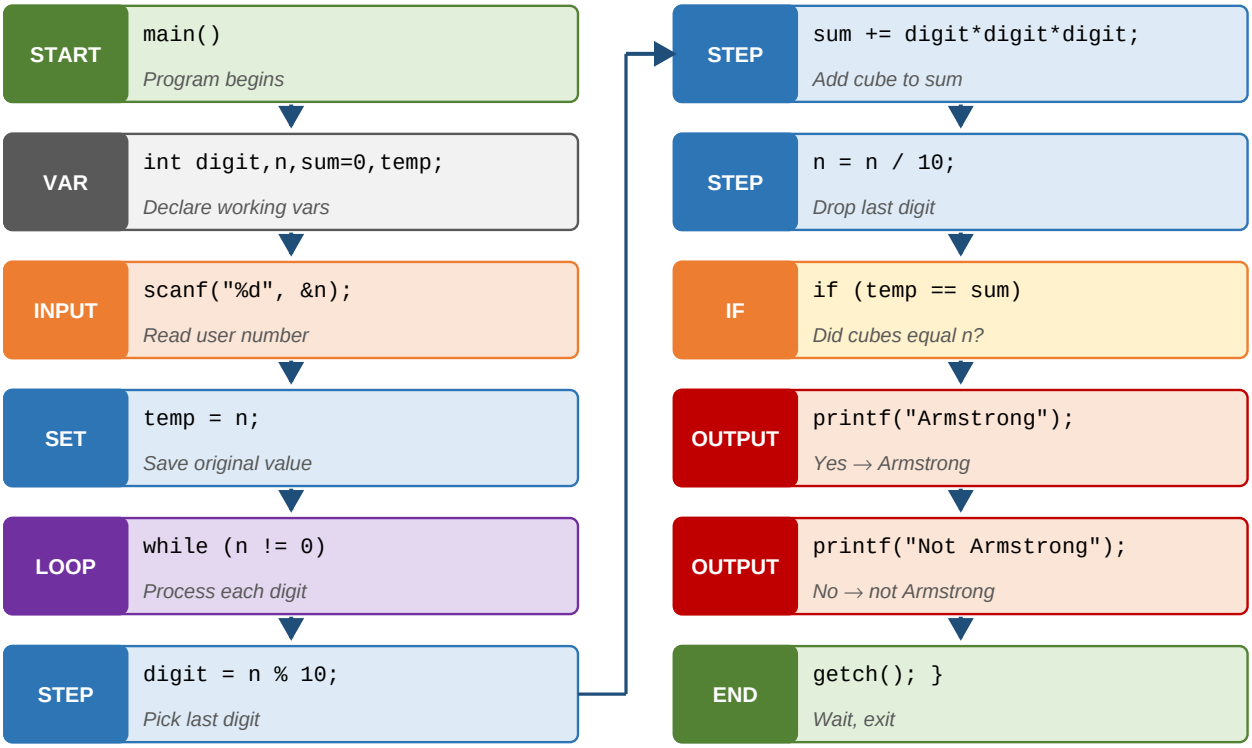
Step 3: sum = 1 + 125 + 27 = 153

Step 4: 153 == 153 ✓ → Armstrong!

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int digit, n, sum = 0, temp;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    temp = n;
    while (n != 0)
    {
        digit = n % 10;
        sum   = sum + (digit * digit * digit);
        n     = n / 10;
    }
    if (temp == sum) printf("Armstrong");
    else             printf("Not Armstrong");
    getch();
}
```

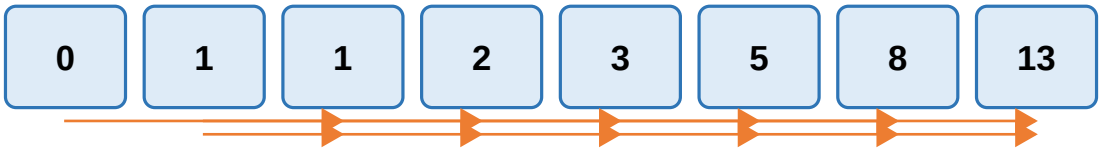
Instruction-by-instruction block diagram



Q11. C program to generate first n terms of Fibonacci sequence

Fibonacci series : each number is the sum of the previous two. Starts with 0 and 1.

Fibonacci Series (first = 0, second = 1)

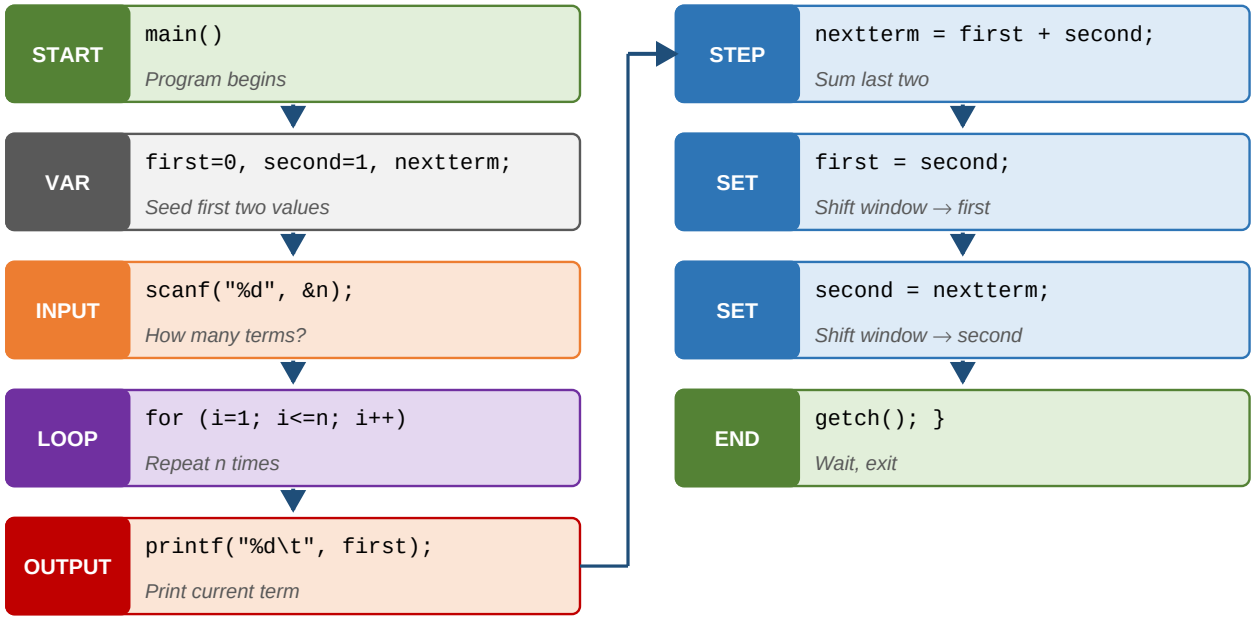


Rule: next = first + second (shift the window by one every time)

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i, n, first = 0, second = 1, nextterm;
    clrscr();
    printf("Enter the number of terms\n");
    scanf("%d", &n);
    printf("Fibonacci Series is\n");
    for (i = 1; i <= n; i++)
    {
        printf("%d\t", first);
        nextterm = first + second;
        first    = second;
        second   = nextterm;
    }
    getch();
}
```

Instruction-by-instruction block diagram



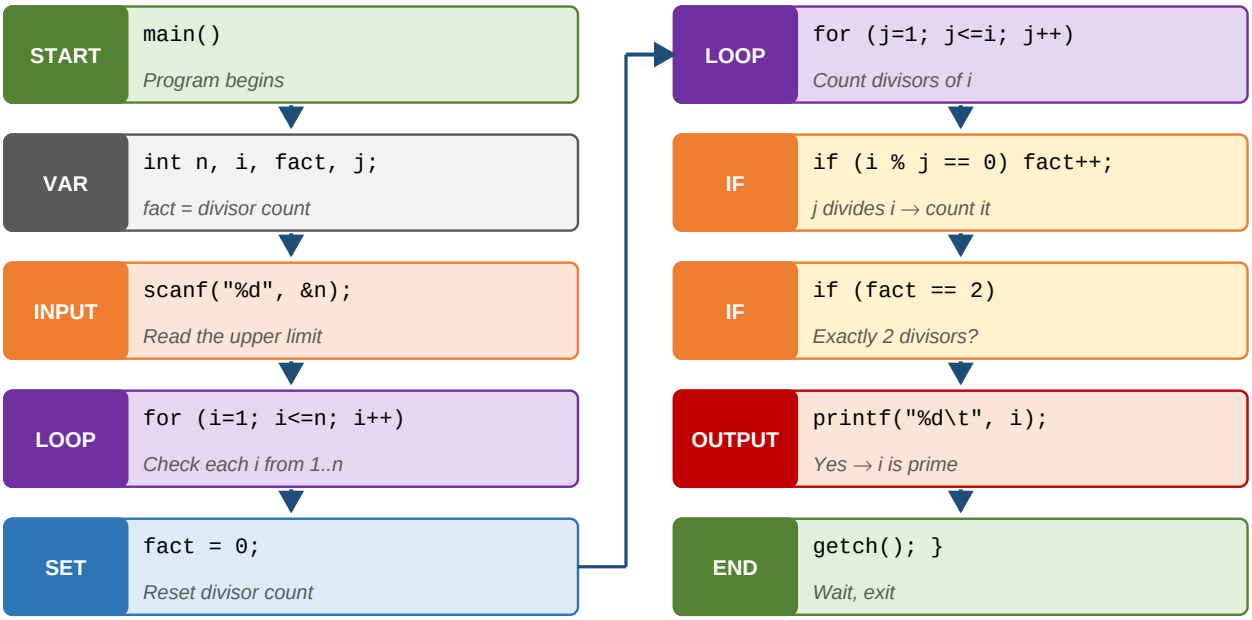
Q12. C program to generate Prime numbers from 1 to n

A **prime number** has exactly two divisors: 1 and itself (2, 3, 5, 7, 11, ...). We count divisors of each number from 1 to n; if the count is 2, it is prime.

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n, i, fact, j;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    printf("Prime numbers from 1 to n are\n");
    for (i = 1; i <= n; i++)
    {
        fact = 0;                          // count divisors
        for (j = 1; j <= i; j++)
            if (i % j == 0) fact++;
        if (fact == 2) printf("%d\t", i);
    }
    getch();
}
```

Instruction-by-instruction block diagram



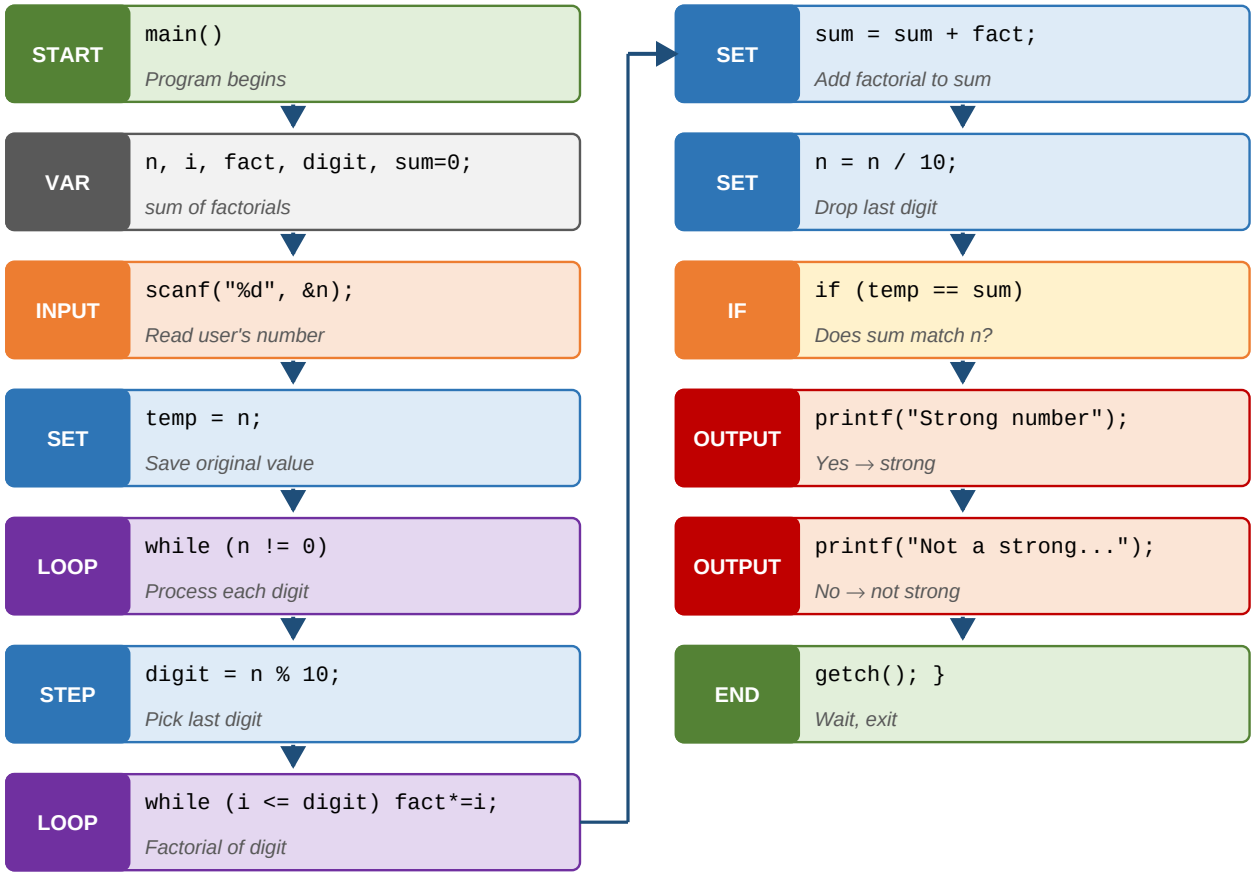
Q13. C program to check whether the given number is Strong

A **Strong number** equals the sum of factorials of its digits. Example: 145 → 1! + 4! + 5! = 1 + 24 + 120 = 145.

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n, i, fact, digit, sum = 0, temp;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    temp = n;
    while (n != 0)
    {
        i = 1; fact = 1;
        digit = n % 10;
        while (i <= digit)         // factorial of digit
        {
            fact = fact * i;
            i++;
        }
        sum = sum + fact;
        n = n / 10;
    }
    if (temp == sum) printf("Strong number");
    else             printf("Not a strong number");
    getch();
}
```

Instruction-by-instruction block diagram



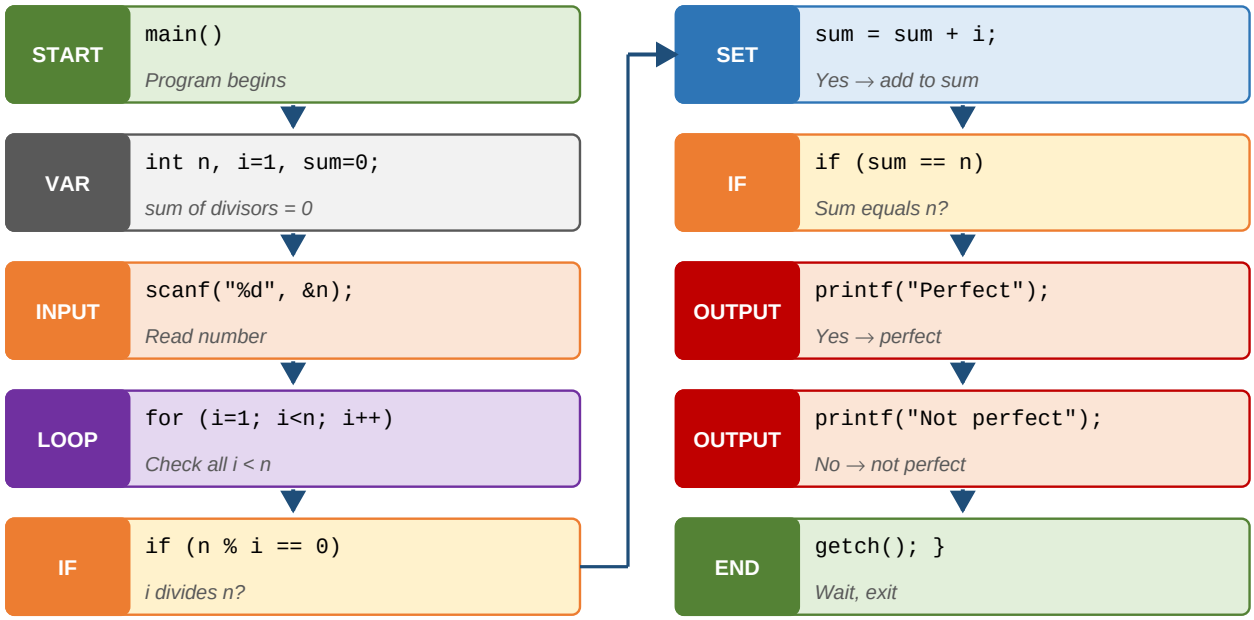
Q14. C program to check whether the given number is Perfect

A **Perfect number** equals the sum of its proper divisors (excluding itself). Example: 6 → 1 + 2 + 3 = 6.

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n, i = 1, sum = 0;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    for (i = 1; i < n; i++)
        if (n % i == 0) sum = sum + i;
    if (sum == n) printf("Perfect number");
    else          printf("Not a perfect number");
    getch();
}
```

Instruction-by-instruction block diagram



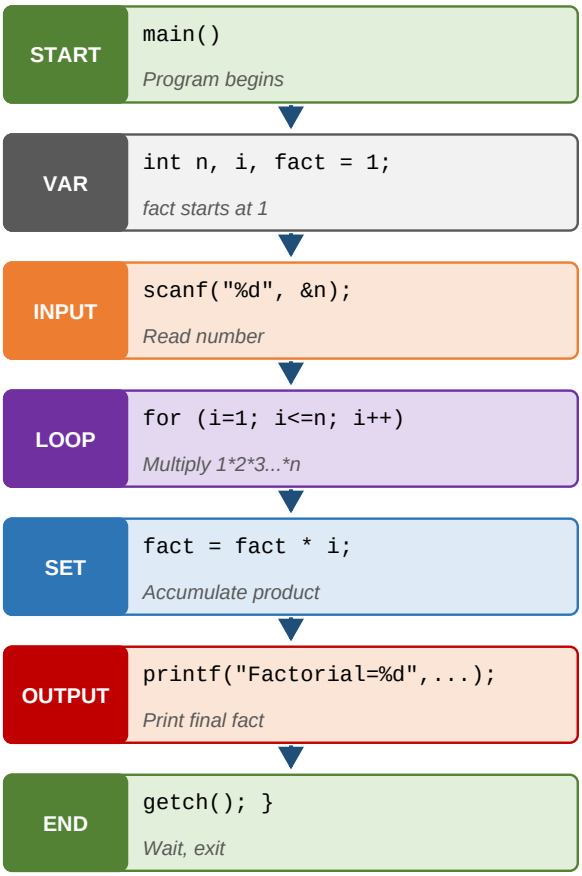
Q15. C program to find the Factorial of a given number

Factorial of n is the product of all integers from 1 to n. 5! = 1 × 2 × 3 × 4 × 5 = 120.

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n, i, fact = 1;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        fact = fact * i;
    printf("Factorial = %d", fact);
    getch();
}
```

Instruction-by-instruction block diagram

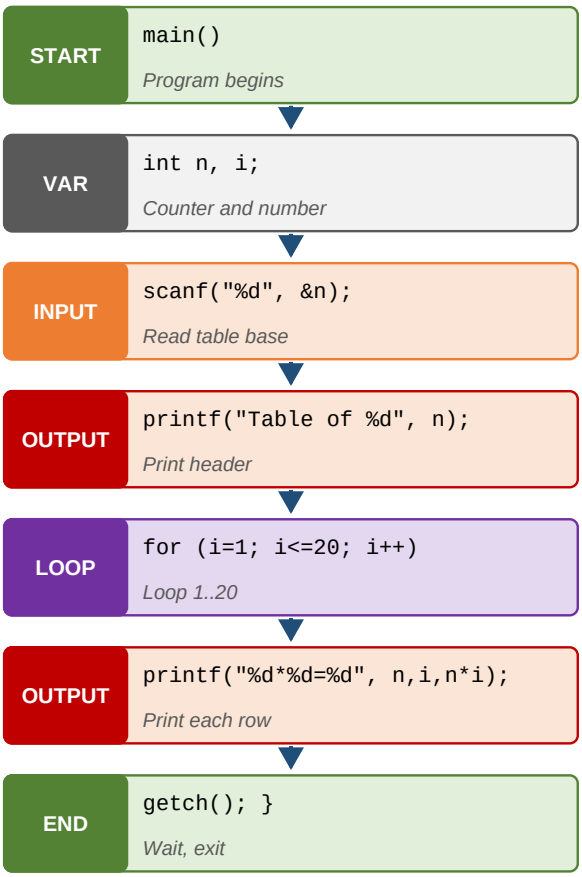


Q16. C program to print the Multiplication Table

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n, i;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    printf("Multiplication table of %d\n", n);
    for (i = 1; i <= 20; i++)
        printf("%d * %d = %d\n", n, i, n * i);
    getch();
}
```

Instruction-by-instruction block diagram



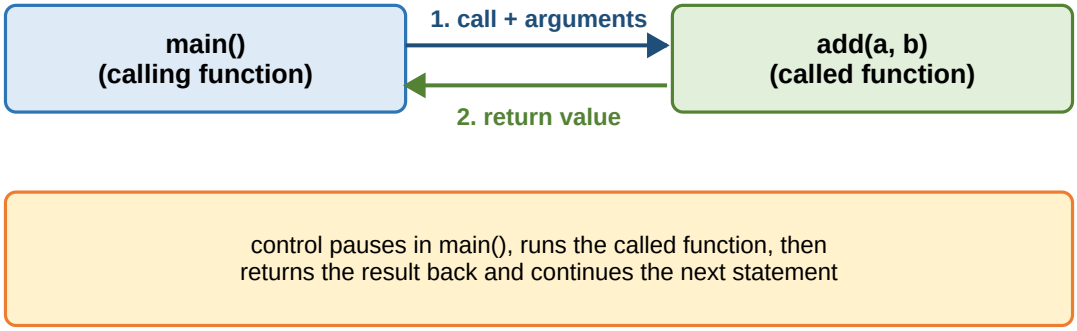
UNIT 3 - Functions

Q1. Define a Function and explain types of functions with example

Simple definition

A **function** is a small sub-program that performs one special task. Splitting big programs into functions makes them easier to read, test, and reuse.

How a Function Call Works



Two types of functions

- **Library (standard) functions** - pre-defined in C. Ex: `printf, scanf, sqrt, pow`.
- **User defined functions** - written by us. Ex: `main, add, factorial`.

Syntax of a function

Syntax

```
returntype functionname(datatype arg1, datatype arg2, ...)
{
    body of function;
    return (expression);
}
```

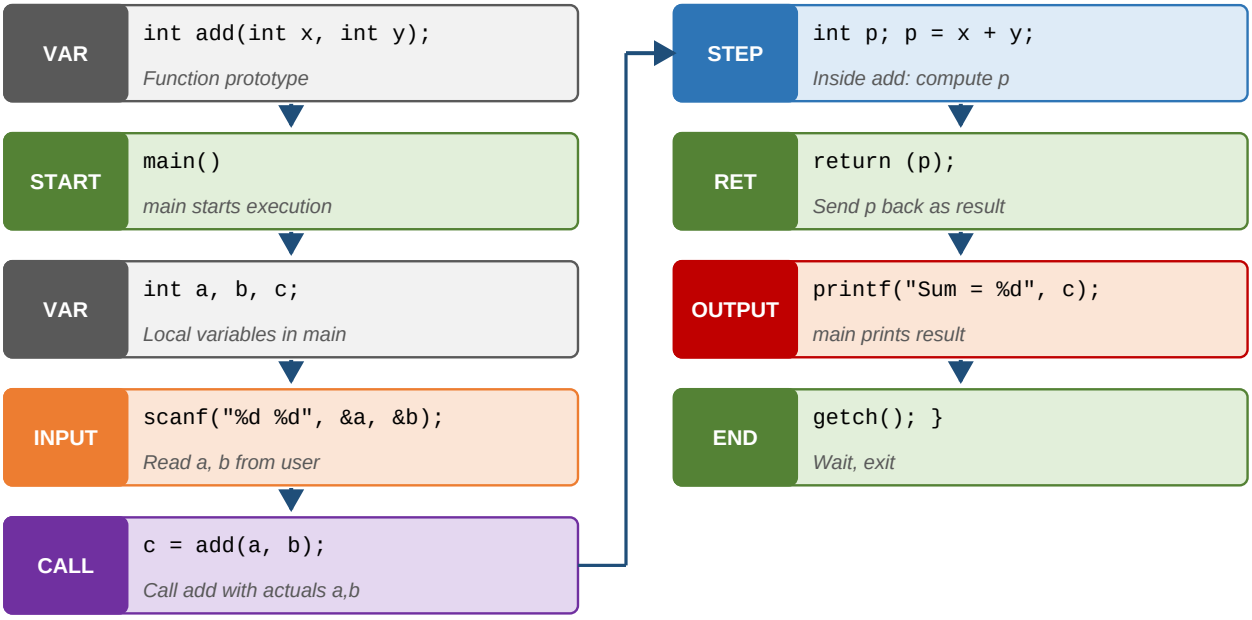
Actual vs formal arguments

- **Actual arguments** - the values passed by the calling function. Ex: `add(A, B)`.
- **Formal arguments** - the variables that receive them in the called function. Ex: `int add(int x, int y)`.

Example Program : add two numbers

```
#include <stdio.h>
#include <conio.h>
int add(int x, int y);   /* prototype */
void main()
{
    int a, b, c;
    clrscr();
    printf("Enter a, b\n");
    scanf("%d %d", &a, &b);
    c = add(a, b);
    printf("Sum = %d", c);
    getch();
}
int add(int x, int y)
{
    int p;
    p = x + y;
    return (p);
}
```

Instruction-by-instruction block diagram



Q2. Categories of Functions (based on arguments and return values)

1. Functions **with arguments** and **with return** values.
2. Functions **without arguments** and **without return** values.
3. Functions **without arguments** and **with return** value.
4. Functions **with arguments** and **without return** values.

Type 1 : With args & with return

Program

```
#include <stdio.h>
#include <conio.h>
int add(int x, int y);
void main()
{
    int a, b, c;
    clrscr();
    printf("Enter a, b\n");
    scanf("%d %d", &a, &b);
    c = add(a, b);
    printf("Sum = %d", c);
    getch();
}
int add(int x, int y) { return (x + y); }
```

Type 2 : No args & no return

Program

```
#include <stdio.h>
#include <conio.h>
void add();
void main() { clrscr(); add(); getch(); }
void add()
{
    int a, b;
    printf("Enter a, b\n");
    scanf("%d %d", &a, &b);
    printf("Sum = %d", a + b);
}
```

Type 3 : No args & with return

Program

```
int add();
void main()
{
    int c = add();
    printf("Sum = %d", c);
}
int add()
{
    int a, b;
    scanf("%d %d", &a, &b);
    return (a + b);
}
```

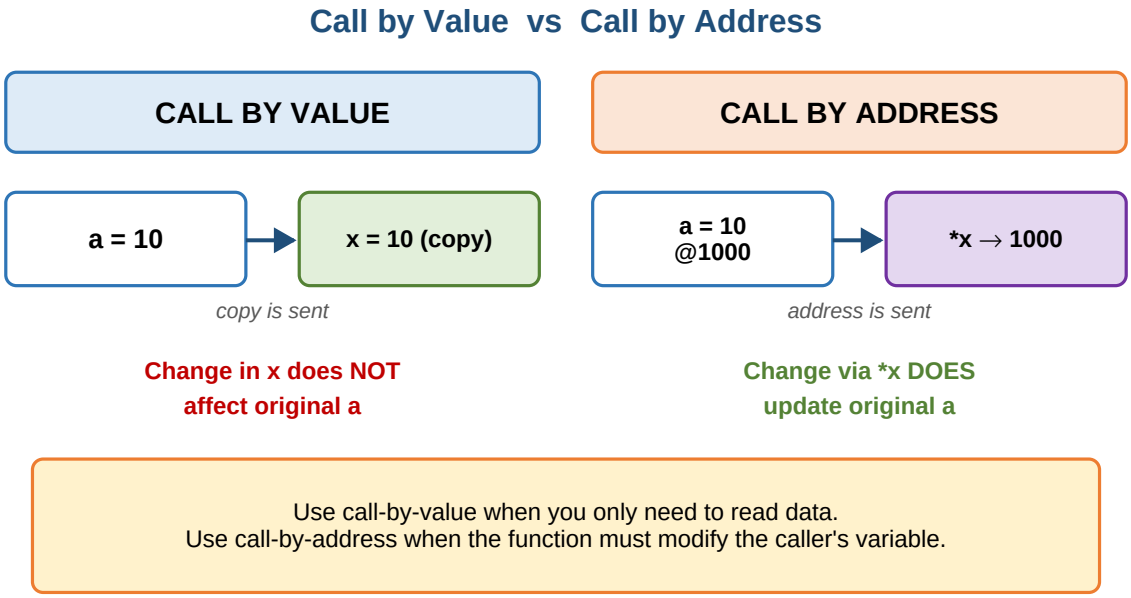
Type 4 : With args & no return

Program

```
void add(int x, int y);
void main()
{
    int a, b;
    scanf("%d %d", &a, &b);
    add(a, b);
}
void add(int x, int y) { printf("Sum = %d", x + y); }
```

Q3. Parameter Passing Techniques with example programs

Parameter passing is how the **calling function** shares data with the **called function**. C has two techniques.



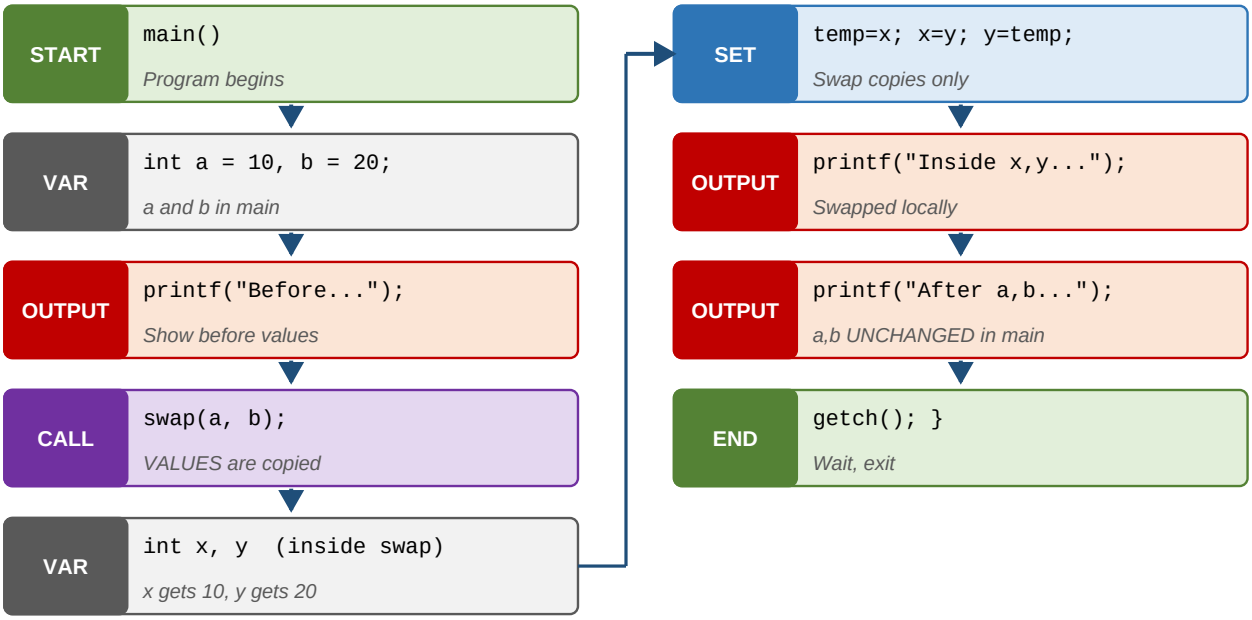
1. Call by Value

The **values** are sent. Changes made inside the called function stay local - the caller's variables are not affected.

Call by Value

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a = 10, b = 20;
    printf("Before: a = %d, b = %d\n", a, b);
    swap(a, b);                          // values passed
    printf("After : a = %d, b = %d\n", a, b);
    getch();
}
int swap(int x, int y)
{
    int temp;
    temp = x; x = y; y = temp;
    printf("Inside swap: x = %d, y = %d\n", x, y);
}
```

Instruction-by-instruction block diagram



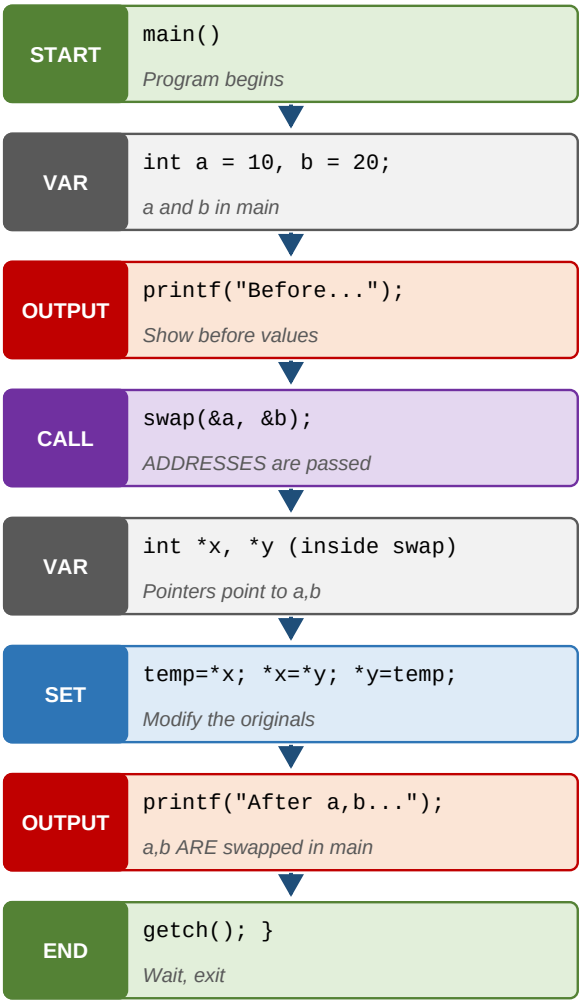
2. Call by Address (Reference)

The **addresses** are sent using `&`. The called function uses pointers `*x, *y` to change the caller's variables directly.

Call by Address

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a = 10, b = 20;
    printf("Before: a = %d, b = %d\n", a, b);
    swap(&a, &b);                          // addresses passed
    printf("After : a = %d, b = %d\n", a, b);
    getch();
}
int swap(int *x, int *y)
{
    int temp;
    temp = *x; *x = *y; *y = temp;
}
```

Instruction-by-instruction block diagram



Q4. Different types of Storage Classes in C

Storage class of a variable decides: **where** it is stored, **what** its default value is, **where** it is visible (scope) and **how long** it lives (lifetime).

4 Storage Classes in C

Keyword	Stored In	Scope	Default
auto	memory	local	garbage
register	CPU reg	local	garbage
static	memory	local	zero
extern	memory	global	zero

Storage class decides WHERE, HOW LONG, and WHO can see a variable.

auto: Default for local variables. Stored in memory. Local scope. Default value = garbage. Lifetime = within the function.

register: Stored in CPU registers (faster, limited). Local scope. Default = garbage.

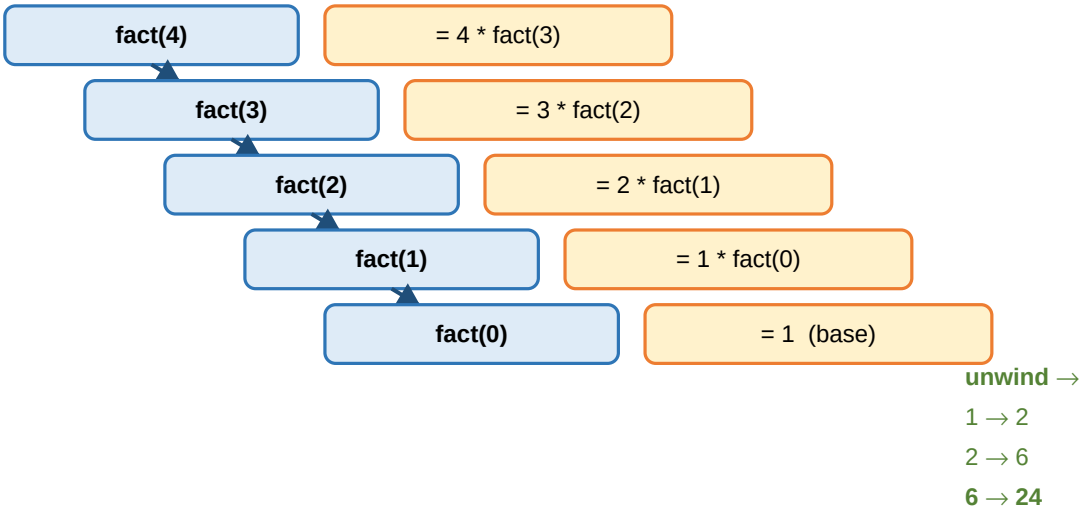
static: Stored in memory. Local scope but value **remembered** between calls. Default = 0.

extern: Stored in memory. **Global scope** across files. Default = 0. Lives until program ends.

Q5. Explain Recursion + C program to find Factorial using Recursion

Recursion is when a function **calls itself** to solve a smaller version of the same problem. Every recursive function must have a **base case** to stop.

Recursion - factorial(4) Unfolded

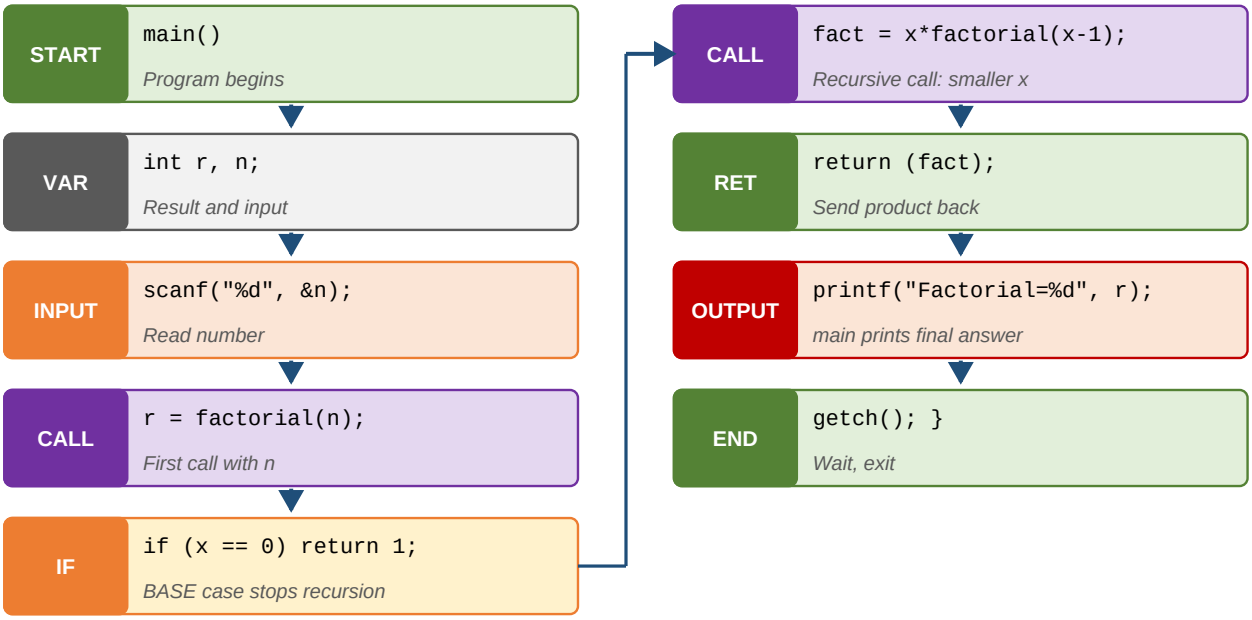


Every recursive function MUST have a base case to stop.

Factorial using Recursion

```
#include <stdio.h>
#include <conio.h>
int factorial(int x);
void main()
{
    int r, n;
    clrscr();
    printf("Enter any number\n");
    scanf("%d", &n);
    r = factorial(n);
    printf("Factorial = %d", r);
    getch();
}
int factorial(int x)
{
    int fact;
    if (x == 0) return (1);                  // base case
    else
    {
        fact = x * factorial(x - 1);         // recursive call
        return (fact);
    }
}
```

Instruction-by-instruction block diagram



Step-by-step explanation

Step 1. Read a number **n** from the user.

Step 2. Call `factorial(n)`.

Step 3. Inside factorial, if **x == 0** return 1 (base case).

Step 4. Otherwise return `x * factorial(x-1)`.

Step 5. The calls stack up, hit the base case, then unwind multiplying all values.

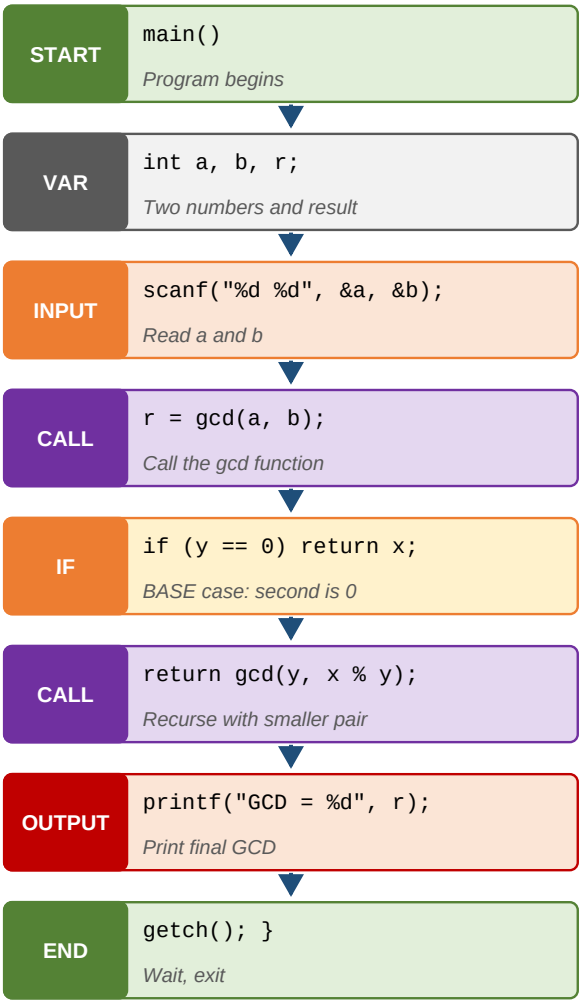
Q6. Explain Recursion + C program to find GCD using Recursion

GCD (Greatest Common Divisor) of two numbers is the largest number that divides both. Using **Euclid's algorithm**: GCD(a, b) = GCD(b, a mod b), and GCD(a, 0) = a.

GCD using Recursion

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a, b, r;
    clrscr();
    printf("Enter two numbers\n");
    scanf("%d %d", &a, &b);
    r = gcd(a, b);
    printf("GCD = %d", r);
    getch();
}
int gcd(int x, int y)
{
    if (y == 0) return (x);            // base case
    else        return gcd(y, x % y);  // recursive call
}
```

Instruction-by-instruction block diagram



UNIT 4 - Arrays and Strings

Q1. Define Array. Explain One Dimensional Array - Declaration and Initialization

Meaning

- An **array** is a collection of elements of the **same type**.
- Elements are stored in **consecutive memory**.
- Array index starts at **0** and ends at **size-1**.
- Arrays give **one name** for many values - no need to declare `x1, x2, x3 ...`.

One Dimensional Array : int arr[5]

Index starts at 0 and goes up to size - 1. Elements are stored in consecutive memory.

Declaration

Syntax

```
datatype arrayname[size];
```

Example: `int arr[10];` reserves space for 10 integers.

Initialization

Syntax

```
datatype arrayname[size] = { v1, v2, ... , vn };
```

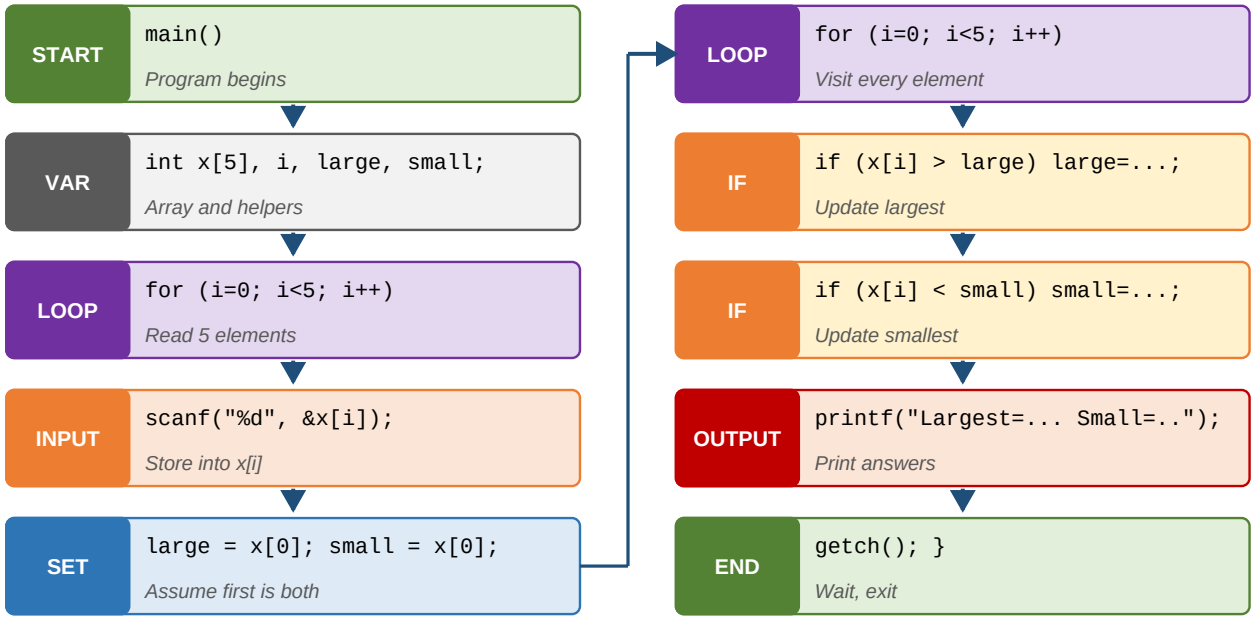
Example: `int x[5] = {10, 20, 30, 40, 50};`

Q2. C program to find the Largest and Smallest elements in an array

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int x[5], i, large, small;
    clrscr();
    printf("Enter 5 integer elements\n");
    for (i = 0; i < 5; i++)
        scanf("%d", &x[i]);
    large = x[0];
    small = x[0];
    for (i = 0; i < 5; i++)
    {
        if (x[i] > large) large = x[i];
        if (x[i] < small) small = x[i];
    }
    printf("Largest  = %d\nSmallest = %d", large, small);
    getch();
}
```

Instruction-by-instruction block diagram



Step-by-step explanation

Step 1. Declare an array of 5 integers and variables **large**, **small**.

Step 2. Read 5 numbers using a for loop.

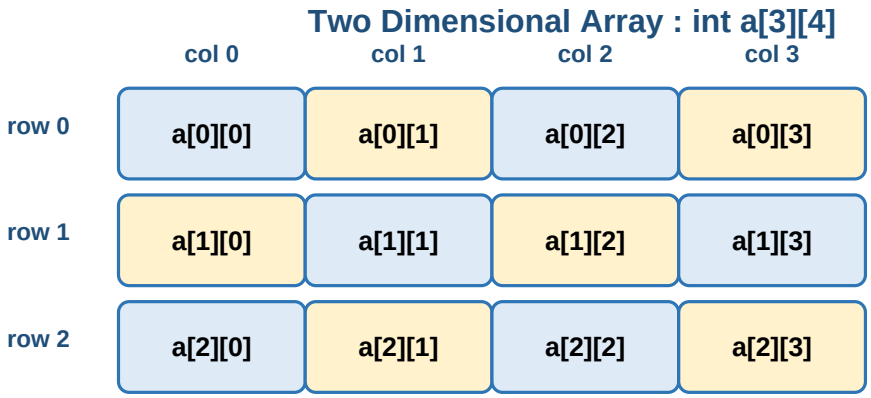
Step 3. Assume the first element is both the largest and smallest.

Step 4. Loop again: update **large** when a bigger value is found, update **small** when a smaller value is found.

Step 5. Print the results.

Q3. Two Dimensional Arrays - Declaration and Initialization

A **2D array** is like a **table** with rows and columns. It is used to store matrices.



3 rows × 4 cols = 12 cells. Access: a[row][column]

Declaration

Syntax

```
datatype arrayname[rows][cols];
```

Initialization

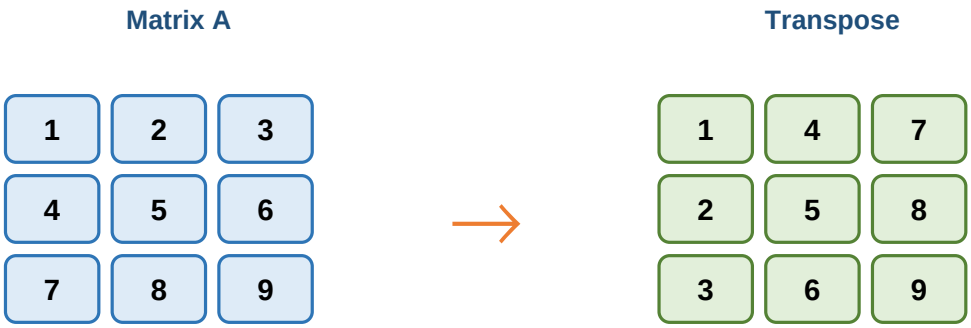
Example

```
int x[2][2] = {10, 20, 30, 40};
```

Q4. C program to perform Transpose of a 3x3 Matrix

Transpose means **swapping rows with columns**. `T[i][j] = A[j][i]`.

Matrix Transpose (swap rows and columns)

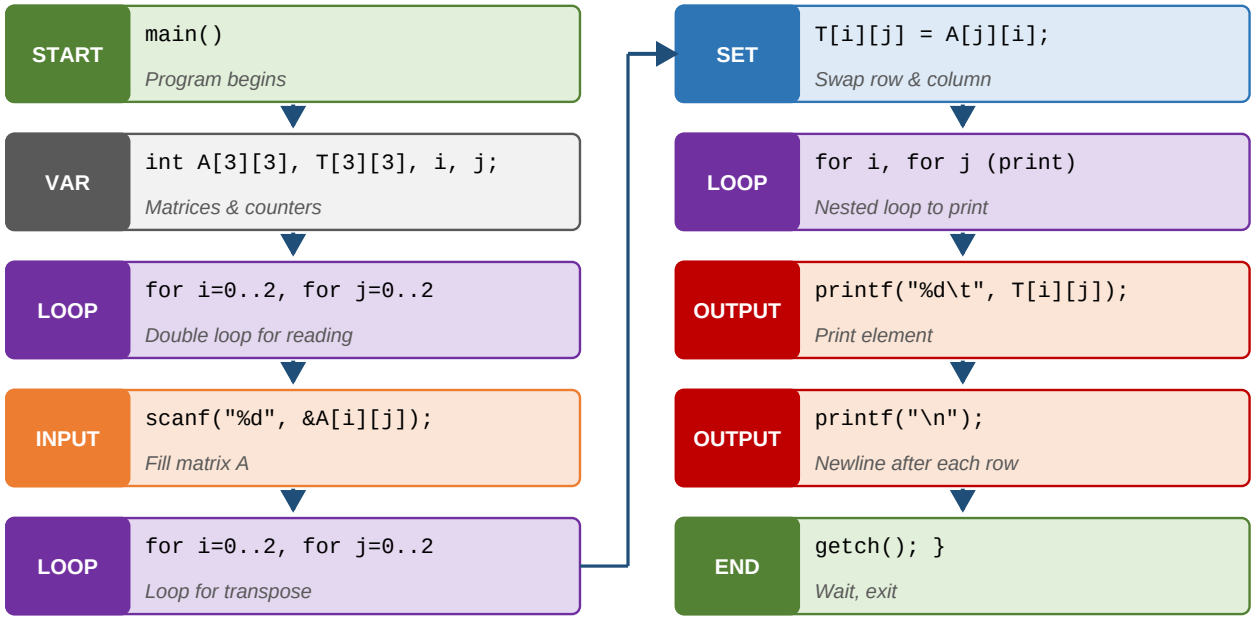


Rule: Transpose[i][j] = A[j][i] (row ↔ column)

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int A[3][3], T[3][3], i, j;
    clrscr();
    printf("Enter elements of matrix A\n");
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            scanf("%d", &A[i][j]);
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            T[i][j] = A[j][i];          // swap i and j
    printf("Transpose of A is\n");
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < 3; j++)
            printf("%d\t", T[i][j]);
        printf("\n");
    }
    getch();
}
```

Instruction-by-instruction block diagram



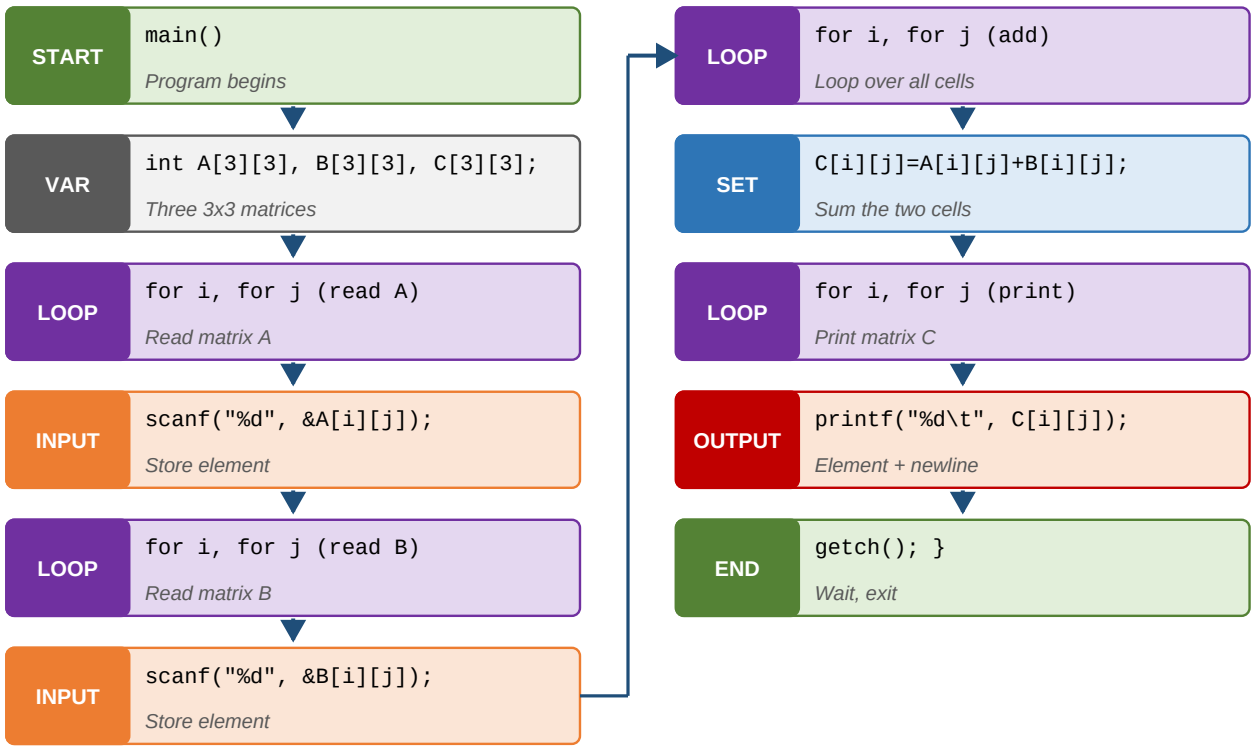
Q5. C program to perform Addition of Two 3x3 Matrices

Add element-by-element: `C[i][j] = A[i][j] + B[i][j]`.

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int A[3][3], B[3][3], C[3][3], i, j;
    clrscr();
    printf("Enter A\n");
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++) scanf("%d", &A[i][j]);
    printf("Enter B\n");
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++) scanf("%d", &B[i][j]);
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            C[i][j] = A[i][j] + B[i][j];
    printf("Result matrix C is\n");
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < 3; j++) printf("%d\t", C[i][j]);
        printf("\n");
    }
    getch();
}
```

Instruction-by-instruction block diagram



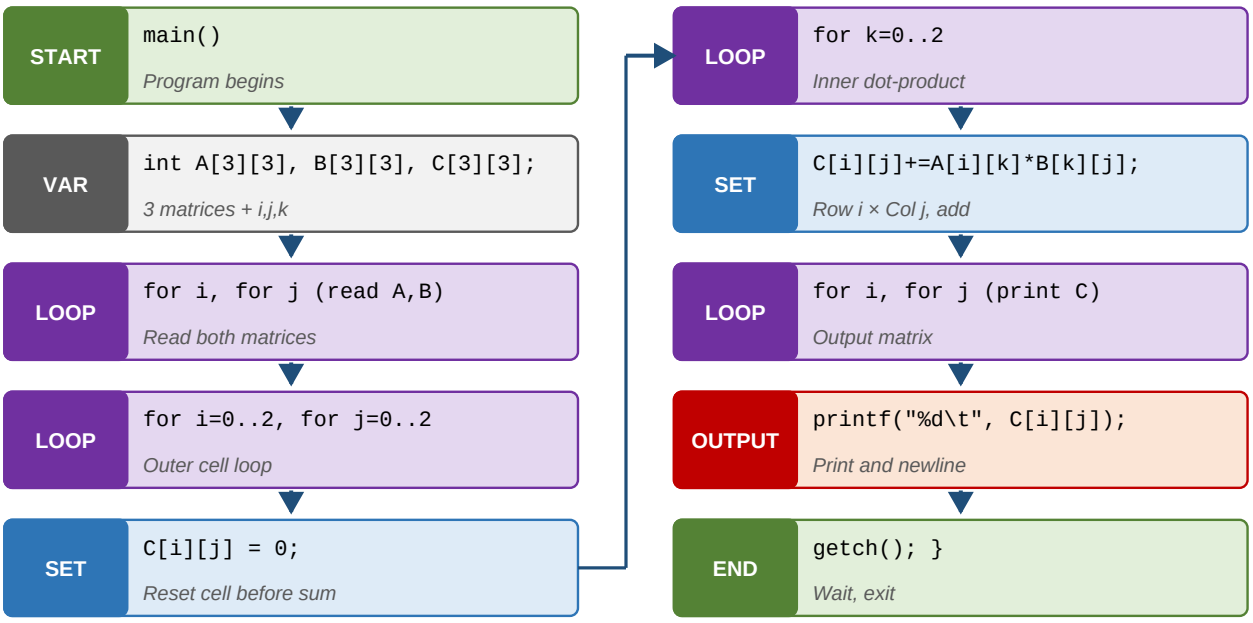
Q6. C program to perform Multiplication of Two 3x3 Matrices

For every cell `C[i][j]`, multiply row **i** of A with column **j** of B, and add up the products.

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int A[3][3], B[3][3], C[3][3], i, j, k;
    clrscr();
    printf("Enter A\n");
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++) scanf("%d", &A[i][j]);
    printf("Enter B\n");
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++) scanf("%d", &B[i][j]);
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
        {
            C[i][j] = 0;
            for (k = 0; k < 3; k++)
                C[i][j] = C[i][j] + (A[i][k] * B[k][j]);
        }
    printf("Result matrix C is\n");
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < 3; j++) printf("%d\t", C[i][j]);
        printf("\n");
    }
    getch();
}
```

Instruction-by-instruction block diagram

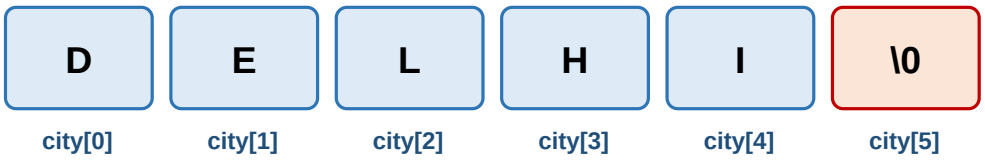


Q7. Define String. Declaration and Initialization in C

Important points

- A string is a group of characters inside **double quotes**.
- A string is also called a **character array**.
- Every string ends with a **null character** `'\0'`.
- Size = number of characters + 1 (for the null).

String Layout : char city[6] = "DELHI"



A string ALWAYS ends with '\0' (null character). Size = number of letters + 1.

Declaration

Syntax

```
char name[20];
```

Initialization

Two ways

```
char city[6] = "DELHI";
        or
char city[6] = {'D','E','L','H','I','\0'};
```

Q8. String Handling Functions in C with example programs

All string functions are defined in <string.h>. Here are the 7 most used ones:

Function	Use
strcpy(s1, s2)	Copies s2 into s1
strcat(s1, s2)	Appends s2 to the end of s1
strrev(s)	Reverses the string s
strlen(s)	Returns length of s (excluding '\0')
strlwr(s)	Converts s to lowercase
strupr(s)	Converts s to UPPERCASE
strcmp(s1, s2)	Compares two strings; 0 if equal

Example : strcpy - copy one string into another

strcpy()

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
{
    char name1[20], name2[20];
    clrscr();
    printf("Enter first name\n");   gets(name1);
    printf("Enter second name\n");  gets(name2);
    strcpy(name1, name2);              // copy name2 into name1
    printf("After copy: %s, %s", name1, name2);
    getch();
}
```

Example : strcat - concatenate two strings

strcat()

```
#include <string.h>
void main()
{
    char a[20], b[20];
    gets(a); gets(b);
    strcat(a, b);                      // b is added to end of a
    printf("%s", a);
}
```

Example : strlen - length of a string

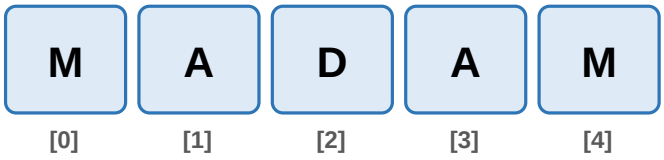
strlen()

```
#include <string.h>
void main()
{
    char name[20];
    int x;
    gets(name);
    x = strlen(name);                  // length without \0
    printf("Length = %d", x);
}
```

Q9. C program to check whether the given string is Palindrome

A **palindrome** reads the same forwards and backwards. Example: MADAM, LEVEL, 12321.

Palindrome Check of "MADAM"

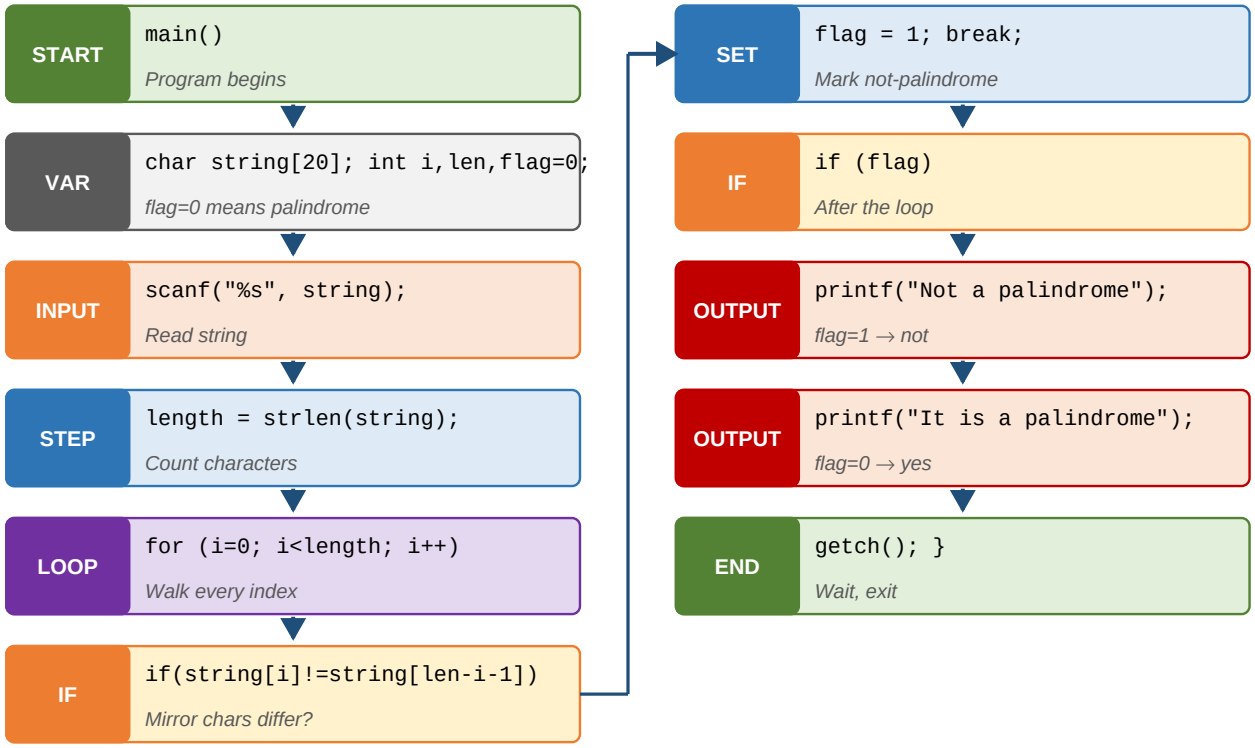


Compare first with last, second with second-last, ... All match → Palindrome!

Program

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
{
    char string[20];
    int i, length, flag = 0;
    clrscr();
    printf("Enter any string\n");
    scanf("%s", &string);
    length = strlen(string);
    for (i = 0; i < length; i++)
    {
        if (string[i] != string[length - i - 1])
        {
            flag = 1;
            break;
        }
    }
    if (flag) printf("Not a palindrome");
    else      printf("It is a palindrome");
    getch();
}
```

Instruction-by-instruction block diagram



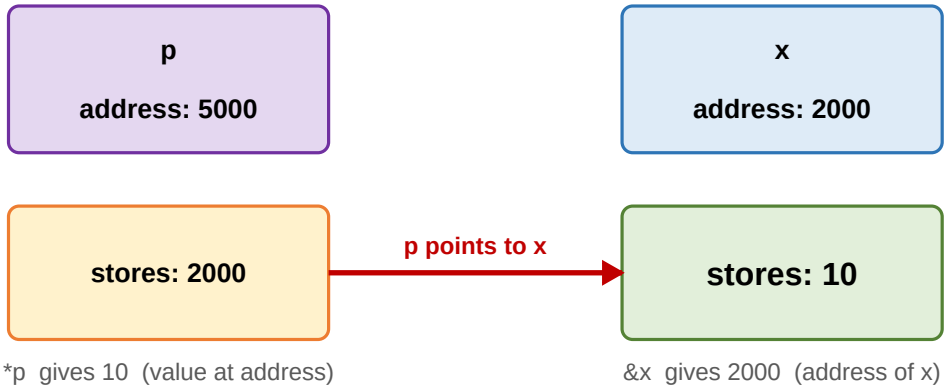
UNIT 5 - Pointers, Structures and Unions

Q1. Define Pointer. Declaration and Initialization with example

Simple explanation

Every variable lives at some **memory address**. A **pointer** is a variable that stores the **address** of another variable. Think of it as a **treasure map** pointing to where the treasure (value) is kept.

Pointer : int x = 10; int *p = &x;



Declaration

Syntax

```
datatype *variable;
```

Example: `int *p;` (p is a pointer that can point to an int).

Two important operators

- **& (address-of)**: returns the address of a variable. Ex: `&x`.
- *** (value-at)**: returns the value stored at a pointer. Ex: `*p`.

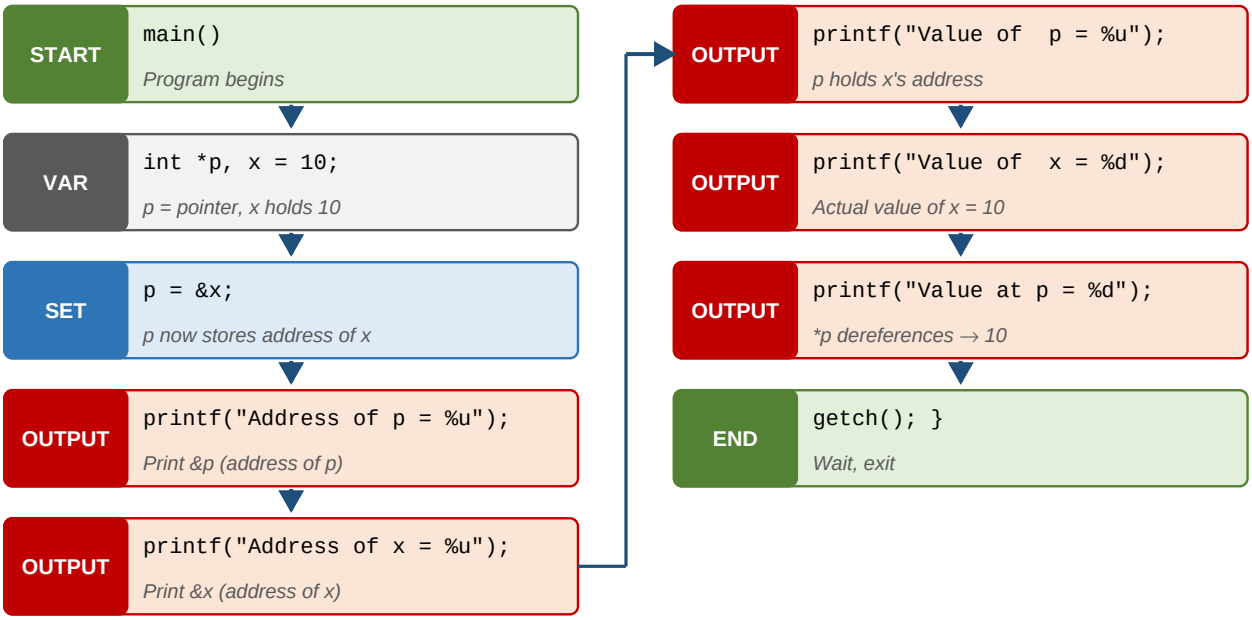
Advantages of pointers

- Direct access to memory - fast.
- Allow functions to return more than one value.
- Reduce storage and execution time.
- Give an alternate way to access array elements.
- Used to pass data back and forth between functions.
- Needed to build linked lists, stacks, queues, trees, graphs.

Pointer example

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int *p, x = 10;
    clrscr();
    p = &x;                                  // p now holds address of x
    printf("Address of p   = %u\n", &p);
    printf("Address of x   = %u\n", &x);
    printf("Value of p     = %u\n",  p);
    printf("Value of x     = %d\n",  x);
    printf("Value at p(*p) = %d\n", *p);
    getch();
}
```

Instruction-by-instruction block diagram



Q2. Explain Pointers with Arrays with an example program

When we declare an array, the compiler allocates memory for all its elements in a row. The **array name** itself is the address of the first element. So `arr` is the same as `&arr[0]`.

One Dimensional Array : int arr[5]

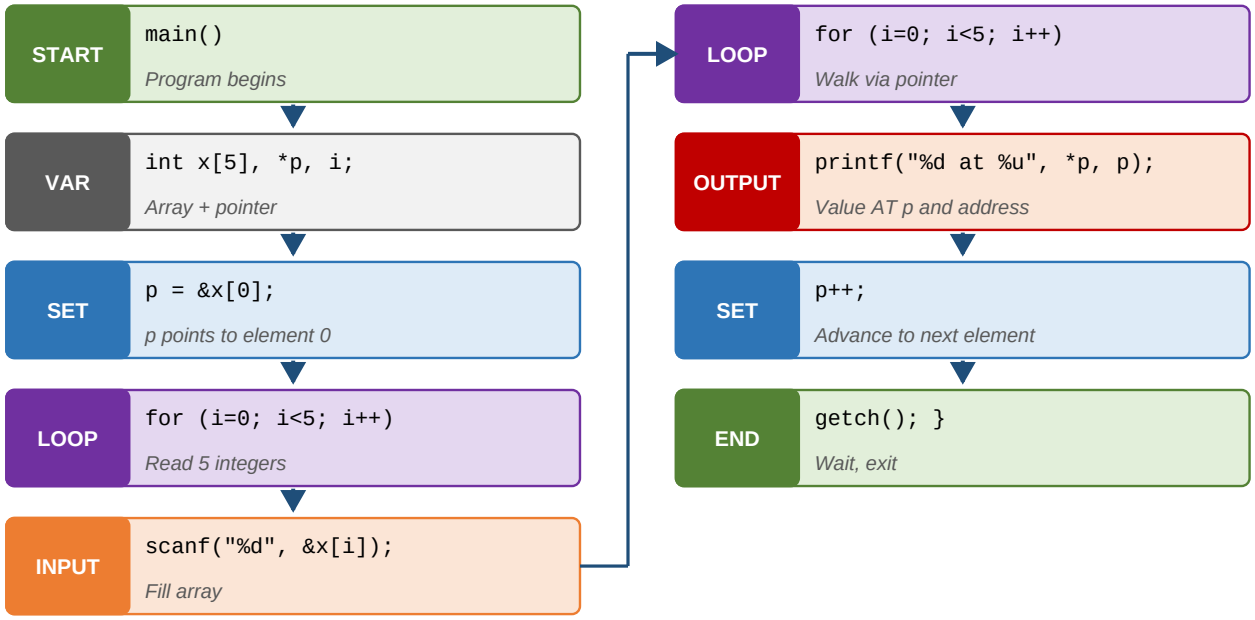
Index starts at 0 and goes up to size - 1. Elements are stored in consecutive memory.

We can make a pointer point to the array and walk through it using p++.

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int x[5], *p, i;
    clrscr();
    p = &x[0];                          // p points to first element
    printf("Enter 5 integers\n");
    for (i = 0; i < 5; i++) scanf("%d", &x[i]);
    printf("Array elements via pointer:\n");
    for (i = 0; i < 5; i++)
    {
        printf("%d stored at %u\n", *p, p);
        p++;                            // move to next element
    }
    getch();
}
```

Instruction-by-instruction block diagram

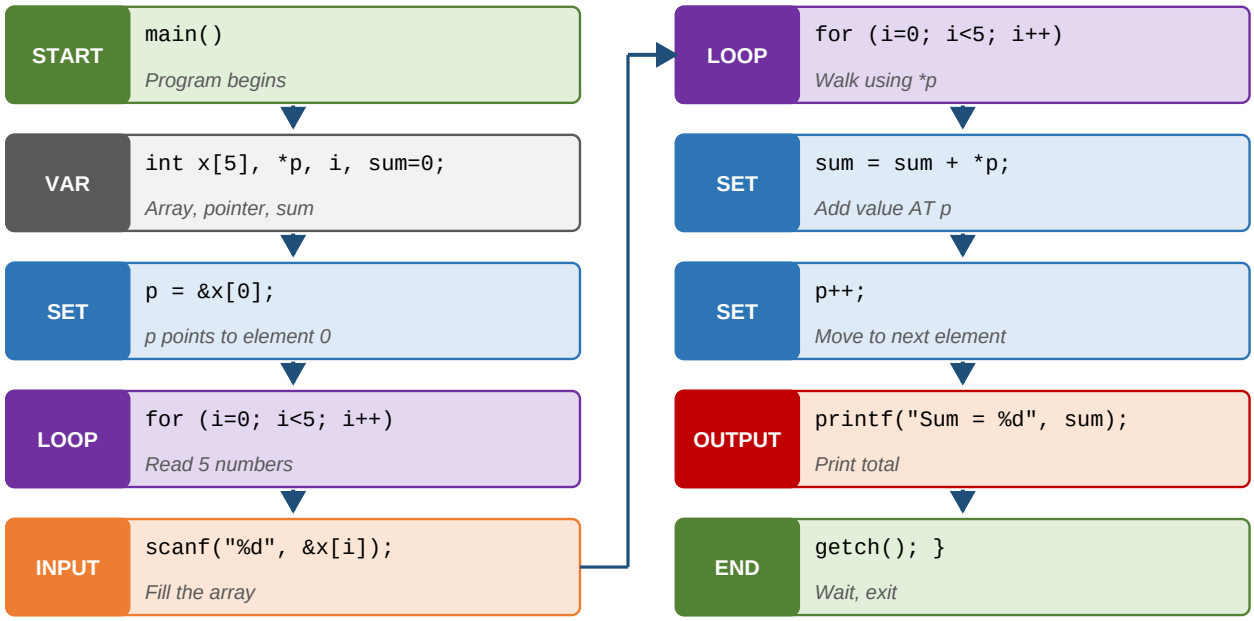


Q3. C program to find the Sum of array elements using Pointers

Program

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int x[5], *p, i, sum = 0;
    clrscr();
    p = &x[0];
    printf("Enter 5 integers\n");
    for (i = 0; i < 5; i++) scanf("%d", &x[i]);
    for (i = 0; i < 5; i++)
    {
        sum = sum + *p;                  // add value pointed to
        p++;                             // move to next element
    }
    printf("Sum = %d", sum);
    getch();
}
```

Instruction-by-instruction block diagram



Step-by-step explanation

Step 1. Declare an int array of size 5, a pointer p, counter i, and sum=0.

Step 2. Set `p = &x[0]` so p points to the first element.

Step 3. Read 5 numbers into the array.

Step 4. Loop 5 times: add `*p` to sum, then `p++` moves to the next element.

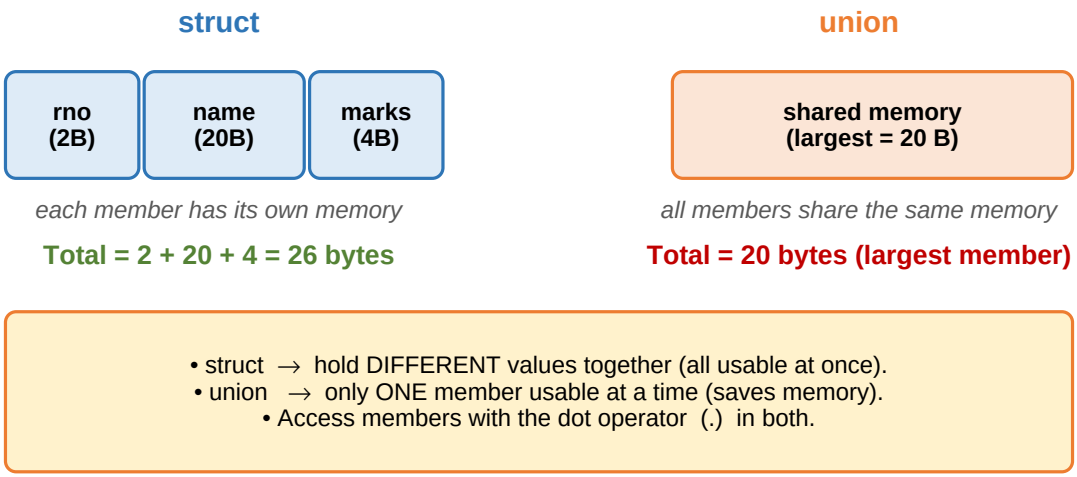
Step 5. Print the sum.

Q4. Define Structure. Declaration, Initialization and Access

Why do we need structures?

Arrays can hold many elements but all of the **same type**. A **structure** can hold elements of **different types** together - perfect for real-world records like *Student (rollno, name, branch, marks)*.

struct vs union - Memory Layout



Declaration

Syntax

```
struct tagname
{
    datatype member1;
    datatype member2;
    ...
};
```

Example

Structure definition

```
struct student
{
    int   rno;
    char  name[20];
    char  branch[20];
    float marks;
};
```

Structure variable

Declaration

```
struct student s;
```

Initialization

Initialization

```
struct student s = { 501, "Dinesh", "CSE", 100.5 };
```

Accessing members - the dot operator

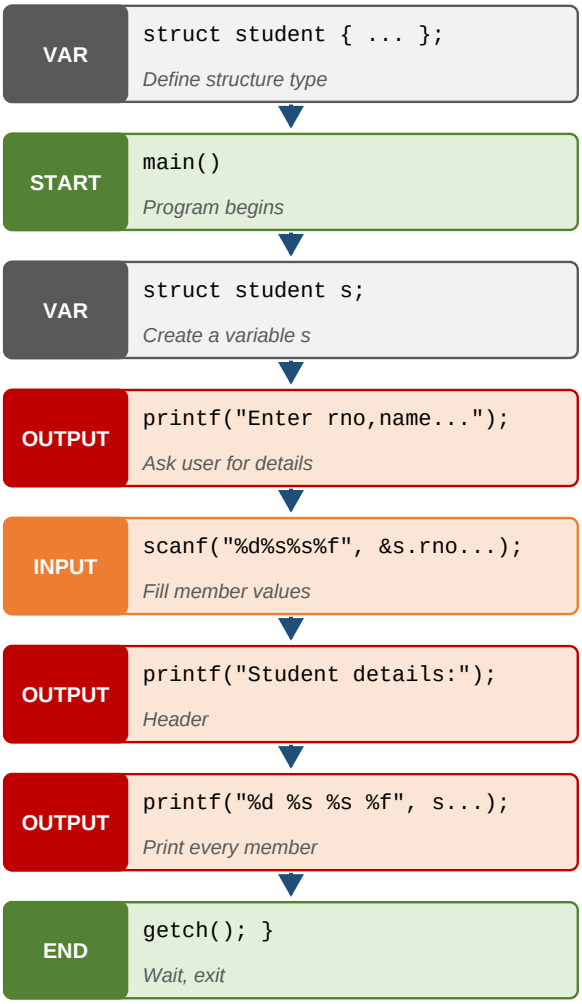
Usage

```
printf("%d", s.rno);
printf("%s", s.name);
printf("%f", s.marks);
```

Structure Program

```
#include <stdio.h>
#include <conio.h>
struct student
{
    int   rno;
    char  name[20];
    char  branch[20];
    float marks;
};
void main()
{
    struct student s;
    clrscr();
    printf("Enter rno, name, branch, marks\n");
    scanf("%d%s%s%f", &s.rno, &s.name, &s.branch, &s.marks);
    printf("Student details:\n");
    printf("%d\t%s\t%s\t%f", s.rno, s.name, s.branch, s.marks);
    getch();
}
```

Instruction-by-instruction block diagram



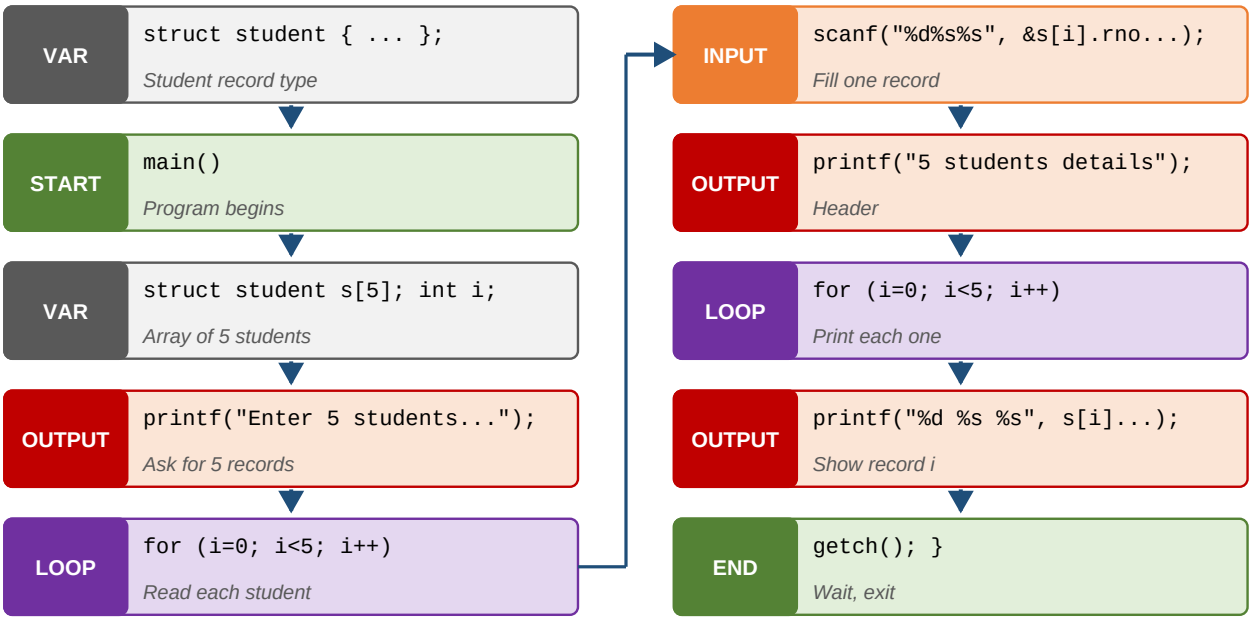
Q5. Array of Structures with an example program

When we need records of **many students** or **many employees**, we use an **array of structures**.

Program

```
#include <stdio.h>
#include <conio.h>
struct student
{
    int  rno;
    char name[20];
    char branch[20];
};
void main()
{
    struct student s[5];
    int i;
    clrscr();
    printf("Enter 5 student details (rno, name, branch)\n");
    for (i = 0; i < 5; i++)
        scanf("%d%s%s", &s[i].rno, &s[i].name, &s[i].branch);
    printf("The 5 students' details are:\n");
    for (i = 0; i < 5; i++)
        printf("%d\t%s\t%s\n", s[i].rno, s[i].name, s[i].branch);
    getch();
}
```

Instruction-by-instruction block diagram



Q6. Differentiate between Structure and Union (with example)

Structure	Union
Each member has its own memory.	All members share the same memory.
Total size = sum of all members.	Total size = size of the largest member.
All members can be used at the same time.	Only one member can be used at a time.
Declared with keyword **struct**.	Declared with keyword **union**.

Union - declaration, initialization, access

Syntax

```
union tagname
{
    datatype m1;
    datatype m2;
    ...
};
union tagname variable;
variable.member = value;
```

Example

Union Example

```
union student
{
    int   rno;
    char  name[20];
    char  branch[20];
    float marks;
};
union student s = { 501, "Dinesh", "CSE", 100.5 };
printf("%s", s.name);   /* only the LAST assignment is safe */
```

Both structure and union use the dot operator `.` to access members.

You did it!

You have walked through every important question in the PPS syllabus - with diagrams, simple English, and step-by-step code explanations. Revisit any section whenever you need a refresher. Practice is the only shortcut to mastery.

Happy Learning!